protpy Descriptor Reference

A complete reference for all protein descriptors available in the **protpy** package, grouped by category.

Composition Descriptors

Amino Acid Composition (AAComp)

Fraction of each of the 20 standard amino acids in the sequence.

```
result = protpy.amino_acid_composition(protein_seq)
# Shape: 1 x 20
# A      C      D      E      F ...
# 6.693  3.108  5.817  3.347  6.614 ...
```

Dipeptide Composition (DPComp)

Frequency of all 400 possible dipeptide (two-residue) combinations.

```
result = protpy.dipeptide_composition(protein_seq)
# Shape: 1 x 400
# AA     AC     AD     AE     AF ...
# 0.797  0.159  0.478  0.399  0.239 ...
```

Tripeptide Composition (TPComp)

Frequency of all 8000 possible tripeptide (three-residue) combinations.

```
result = protpy.tripeptide_composition(protein_seq)
# Shape: 1 x 8000
# AAA    AAC    AAD    AAE    AAF ...
# 0.080  0.000  0.000  0.160  0.000 ...
```

Grand Average of Hydropathy (GRAVY)

Mean of the Kyte-Doolittle hydropathy values across all residues. A positive value indicates overall hydrophobicity; negative indicates overall hydrophilicity.

```
result = protpy.gravy(protein_seq)
# Shape: 1 x 1
# GRAVY
# -0.045
```

Aromaticity

Fraction of aromatic residues (F, W, Y, H) in the sequence.

```
result = protpy.aromaticity(protein_seq)
# Shape: 1 x 1
# Aromaticity
# 0.118
```

Instability Index

> **v1.6.0:** the DIWV table now matches Biopython / ExPASy (25 of the 400 entries were wrong before, e.g. DN = 42.67), so values change.

Stability classifier based on dipeptide instability weight values (DIWV). Values below 40 indicate a stable protein; 40 or above indicates instability.

```
result = protpy.instability_index(protein_seq)
# Shape: 1 x 1
# InstabilityIndex
# 32.419
```

Isoelectric Point

> **v1.6.0:** the default pKa set is now `"bjellqvist"` (as ExPASy ProtParam and Biopython); `pka_scale="protpy"` gives the old values.

Estimated pH at which the protein carries no net charge, found by bisection using the Bjellqvist pKa values for ionisable residues and termini.

```
result = protpy.isoelectric_point(protein_seq)
# Shape: 1 x 1
# IsoelectricPoint
# 5.562
```

Molecular Weight

Average molecular weight of the protein calculated from residue masses, corrected for water lost at each peptide bond.

```
result = protpy.molecular_weight(protein_seq)
# Shape: 1 x 1
# MolecularWeight
# 139125.120
```

Charge Distribution

> **v1.6.0:** the default pKa set is now `"bjellqvist"`; `normalize="length"`/`"mass"` gives charge per residue / per Da (replacing `charge_density`).

Positive, negative, and net charge contributions of ionisable residues at a given pH using the Henderson-Hasselbalch equation.

Parameter	Type	Default	Description
ph	float	7.4	pH at which to calculate charge

```
# Default pH 7.4
result = protpy.charge_distribution(protein_seq)

# Custom pH
result = protpy.charge_distribution(protein_seq, ph=6.0)

# Shape: 1 x 3
# PositiveCharge  NegativeCharge  NetCharge
```

```
# 99.398          114.920         -15.522
```

Hydrophobic/Polar/Charged Composition (HPC)

> **Deprecated in v1.6.0** (removed in 2.0): use `kmer_composition(seq, k=1, alphabet=3)`. P is now in the charged class (it was in no class, so the values summed to 100 − P%).

Percentage of residues belonging to each of three physicochemical groups: hydrophobic (A, C, F, I, L, M, V, W, Y), polar (G, N, Q, S, T), and charged (D, E, H, K, R).

```
result = protpy.hydrophobic_polar_charged_composition(protein_seq)
# Shape: 1 x 3
# Hydrophobic  Polar   Charged
# 44.542       32.669  22.789
```

Secondary Structure Propensity (SSP)

> **Deprecated in v1.6.0** (removed in 2.0): a linear map of the amino acid composition; average an AAindex record with `gravy(seq, scale=...)` instead.

Average Chou-Fasman propensity values for alpha-helix, beta-sheet, and random coil conformations across all residues.

```
result = protpy.secondary_structure_propensity(protein_seq)
# Shape: 1 x 3
# Helix  Sheet  Coil
# 0.983  1.050  1.043
```

k-mer Composition

> **v1.6.0:** `k` is limited to 1-3, and `alphabet=` (size 2/3/4/6, a named alphabet or a list of residue groups) absorbs `reduced_alphabet_composition`. `tripeptide_composition` is an alias of `k=3`.

Frequency of all possible k-length residue subsequences, expressed as a percentage of total k-mers.

Parameter	Type	Default	Description
k	int	2	Length of each k-mer

```
# Default k=2 (dipeptide frequencies)
result = protpy.kmer_composition(protein_seq)

# Custom k
result = protpy.kmer_composition(protein_seq, k=3)

# Shape: 1 x 20^k  (e.g. 1 x 400 for k=2)
# AA     AC     AD     AE     AF ...
# 0.797  0.159  0.478  0.399  0.239 ...
```

Reduced Alphabet Composition

> **Deprecated in v1.6.0** (removed in 2.0): use `kmer_composition(seq, k, alphabet=...)`. An invalid `alphabet_size` now raises.

Amino acid composition after mapping residues to a reduced alphabet of physicochemical groups. Supported alphabet sizes: 2, 3, 4, 6.

Parameter	Type	Default	Description
alphabet_size	int	6	Number of reduced groups

```
# Default alphabet_size=6
result = protpy.reduced_alphabet_composition(protein_seq)

# Custom size
result = protpy.reduced_alphabet_composition(protein_seq, alphabet_size=4)

# Shape: 1 x alphabet_size (1 x 6 by default)
# ReducedAlphabet_1  ReducedAlphabet_2  ReducedAlphabet_3  ReducedAlphabet_4  ReducedAlphabet_5 ...
# 25.339             34.741             9.163              9.084              10.837 ...
```

Motif Composition

> **v1.6.0:** KDEL_retention is anchored to the C-terminus; motifs accepts a list of built-in names (including the PROSITE N_glycosylation, ER_retention, PTS1 and tryptic_site) or patterns; per_100=True adds densities.

Count of occurrences (including overlapping) of biological sequence motifs matched via regular expressions. Eight built-in motifs are used by default; a custom list can be supplied.

Default motifs: | Column | Pattern | Biological meaning | |--------|---------|------------------| | NxST_glycosylation | N[^P][ST] | N-linked glycosylation site | | RGD_integrin | RGD | Integrin-binding RGD motif | | KDEL_retention | KDEL | ER retention signal | | CxxC_zinc_finger | C..C | Zinc-finger CxxC motif | | CAAX_prenylation | C[A-Z]{2}[CSIM]$ | CAAX prenylation box | | cAMP_PKA | [RK]{2}.[ST] | cAMP/PKA phosphorylation site | | dileucine_sorting | [DE]xxxL[LI] | Dileucine lysosomal sorting signal | | PEST_region | P.{1,10}[ED] | PEST degradation signal |

Parameter	Type	Default	Description
motifs	list or None	None	Custom regex patterns; uses built-in 8 if None

```
# Default built-in motifs
result = protpy.motif_composition(protein_seq)

# Custom motif list
result = protpy.motif_composition(protein_seq, motifs=[r'RGD', r'N[^P][ST]'])

# Shape: 1 x len(motifs) (1 x 8 by default)
# NxS/T_glycosylation  RGD_integrin  KDEL_retention  CxxC_zinc_finger  CAAX_prenylation ...
# 23                   0             0               2                 0 ...
```

Amino Acid Pair Composition

> **Deprecated in v1.6.0** (removed in 2.0): the values are exactly dipeptide_composition.

Frequency of all 400 residue-pair combinations with column names annotated by the physicochemical class of each residue (Hydrophobic, Polar, Charged, or Other).

```
result = protpy.amino_acid_pair_composition(protein_seq)
# Shape: 1 x 400
# AA_Hydrophobic-Hydrophobic  AC_Hydrophobic-Hydrophobic  AD_Hydrophobic-Charged ...
# 0.797                       0.159                       0.478                  ...
```

Aliphatic Index

A measure of the relative volume occupied by aliphatic side chains (Ala, Val, Ile, Leu). Higher values indicate greater thermostability. Formula: AI = Ala% + 2.9×Val% + 3.9×(Ile%+Leu%).

```
result = protpy.aliphatic_index(protein_seq)
# Shape: 1 x 1
# AliphaticIndex
# 82.725
```

Extinction Coefficient

The molar extinction coefficient at 280 nm, calculated from the number of Trp (W), Tyr (Y), and Cys (C) residues. Reported for both reduced (no disulfide bonds) and oxidized (all Cys paired) states.

```
result = protpy.extinction_coefficient(protein_seq)
# Shape: 1 x 2
# ExtCoeff_Reduced  ExtCoeff_Oxidized
# 140960            143335
```

Boman Index

Sum of solubility values for amino acids divided by sequence length, predicting potential for protein–protein interactions. Positive values suggest membrane-binding or interaction potential.

```
result = protpy.boman_index(protein_seq)
# Shape: 1 x 1
# BomanIndex
# 1.198
```

Aggregation Propensity

> **Deprecated in v1.6.0** (removed in 2.0): use `aggrescan`. Overlapping windows are now merged, so `AggregProneRegions` counts regions.

Estimates aggregation-prone regions via a sliding-window approach combining Kyte–Doolittle hydrophobicity and charge neutrality. Returns the count of qualifying windows and the fraction of the sequence covered.

Parameter	Type	Default	Description
`window`	int	`5`	Sliding window size
`hydrophobicity_threshold`	float	`2.0`	Minimum mean hydrophobicity
`charge_threshold`	int	`0`	Maximum charged residues per window

```
result = protpy.aggregation_propensity(protein_seq)
# Shape: 1 x 2
# AggregProneRegions  AggregProneFraction
# 22                  11.793
```

Hydrophobic Moment

The mean and maximum hydrophobic moment across sliding windows, using the Eisenberg hydrophobicity scale and a helical-wheel projection. Captures amphipathicity of putative helix segments.

Parameter	Type	Default	Description
`window`	int	`11`	Sliding window size

Parameter	Type	Default	Description
angle	int	100	Residue rotation angle in degrees (100° for α-helix)

```
result = protpy.hydrophobic_moment(protein_seq)
# Shape: 1 x 2
# HydrophobicMoment_Mean  HydrophobicMoment_Max
# 0.272                   0.813
```

Shannon Entropy

> **v1.6.0:** moved to the `complexity` module (still importable from `composition`).

Information-theoretic measure of amino acid diversity in a sequence. Computed as:

$$H = -\sum_i p_i \log_2 p_i$$

where p_i is the fractional frequency of each amino acid type present. A value of 0 indicates a completely repetitive (single amino acid) sequence; the theoretical maximum of $\log_2(20) \approx 4.322$ bits corresponds to a perfectly uniform distribution across all 20 canonical amino acids. Widely used as a low-complexity filter and diversity measure in ML feature pipelines.

Parameter	Type	Default	Description
sequence	str	—	Protein sequence

```
result = protpy.shannon_entropy(protein_seq)
# Shape: 1 x 1
# ShannonEntropy
# 4.163
```

Pseudo Amino Acid Composition (PAAComp)

> **v1.6.0:** `properties` accepts AAindex codes and/or dicts; the default is hydrophobicity, hydrophilicity and side-chain mass; `lamda` must be less than the sequence length.

Augmented amino acid composition that incorporates sequence-order effects via correlation factors derived from physicochemical properties. Reduces the dimensionality problem of pure sequence-order information while retaining more sequence information than simple AAComp.

Parameter	Type	Default	Description
lamda	int	30	Number of sequence-order correlation factors to include
weight	float	0.05	Weighting factor for correlation layers
properties	list	[]	AAIndex accession numbers to use (uses built-in set if empty)

```
# Default parameters
result = protpy.pseudo_amino_acid_composition(protein_seq)

# Custom parameters
result = protpy.pseudo_amino_acid_composition(protein_seq, lamda=10, weight=0.1)

# Shape: 1 x (20 + lamda)  →  1 x 50 with defaults
# PAAC_1  PAAC_2  PAAC_3  PAAC_4  PAAC_5 ...
# 1.871   0.869   1.626   0.935   1.849 ...
```

Amphiphilic Pseudo Amino Acid Composition (APAAComp)

> **v1.6.0:** `properties` accepts AAindex codes and/or dicts. The normalisation term 1 + w·Σtau can approach 0 for short sequences: a `RuntimeWarning` is raised below 0.1 and a `ValueError` at or below 0 (use a smaller `weight` or `lamda`).

Extension of PAAComp that uses both hydrophobicity and hydrophilicity properties to capture amphiphilic patterns (dual hydrophobic/hydrophilic character) along the sequence.

Parameter	Type	Default	Description
`lamda`	int	`30`	Number of sequence-order correlation factors
`weight`	float	`0.5`	Weighting factor for correlation layers
`properties`	list	`[]`	AAIndex accession numbers (defaults to hydrophobicity + hydrophilicity)

```
# Default parameters
result = protpy.amphiphilic_pseudo_amino_acid_composition(protein_seq)

# Custom parameters
result = protpy.amphiphilic_pseudo_amino_acid_composition(protein_seq, lamda=15, weight=0.3)

# Shape: 1 x (20 + 2*lamda)  →  1 x 80 with defaults
# APAAC_1  APAAC_2  APAAC_3  APAAC_4  APAAC_5 ...
# 6.049    2.809    5.257    3.025    5.977 ...
```

Autocorrelation Descriptors

> **v1.6.0:** the three methods are `autocorrelation(seq, method="moreaubroto"/"moran"/"geary")`; the three names below are permanent aliases. Only Geary is in the classic `all_descriptors` set. A `lag` that is not less than the sequence length now raises a `ValueError` (it silently became 30).

Autocorrelation descriptors measure the correlation between physicochemical property values of residues separated by a lag distance along the sequence. By default, 8 AAIndex properties are used, generating `lag × 8 = 240` features.

Default properties: | AAIndex ID | Property | |------------|----------| | `CIDH920105` | Normalised average hydrophobicity | | `BHAR880101` | Average flexibility indices | | `CHAM820101` | Polarizability parameter | | `CHAM820102` | Free energy of solution in water | | `CHOC760101` | Residue accessible surface area in tripeptide | | `BIGC670101` | Residue volume | | `CHAM810101` | Steric parameter | | `DAYM780201` | Relative mutability |

Moreaubroto Autocorrelation (MBAuto)

Uses raw property values as the basis for correlation measurement.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum lag distance
`properties`	list	(8 defaults above)	AAIndex accession numbers
`normalize`	bool	`True`	Normalise output values

```
# Default parameters
result = protpy.moreaubroto_autocorrelation(protein_seq)

# Custom parameters
result = protpy.moreaubroto_autocorrelation(protein_seq, lag=15, properties=["CIDH920105"])
```

```
# Shape: 1 x (lag × len(properties))  →  1 x 240 with defaults
# MBAuto_CIDH920105_1  MBAuto_CIDH920105_2  MBAuto_CIDH920105_3  MBAuto_CIDH920105_4  MBAuto_CIDH920105_5 ...
# -0.052               -0.072               -0.036               -0.015               -0.048 ...
```

Moran Autocorrelation (MAuto)

Uses the deviation from the mean property value, making it mean-centred and thereby less sensitive to the absolute property scale.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum lag distance
`properties`	list	(8 defaults above)	AAIndex accession numbers
`normalize`	bool	`True`	Normalise output values

```
# Default parameters
result = protpy.moran_autocorrelation(protein_seq)

# Custom parameters
result = protpy.moran_autocorrelation(protein_seq, lag=15)

# Shape: 1 x (lag × len(properties))  →  1 x 240 with defaults
# MAuto_CIDH920105_1  MAuto_CIDH920105_2  MAuto_CIDH920105_3  MAuto_CIDH920105_4  MAuto_CIDH920105_5 ...
# -0.058              -0.078              -0.042              -0.021              -0.054 ...
```

Geary Autocorrelation (GAuto)

Uses squared differences between property values at each lag, making it sensitive to local dissimilarities rather than global correlation.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum lag distance
`properties`	list	(8 defaults above)	AAIndex accession numbers
`normalize`	bool	`True`	Normalise output values

```
# Default parameters
result = protpy.geary_autocorrelation(protein_seq)

# Custom parameters
result = protpy.geary_autocorrelation(protein_seq, lag=10, normalize=False)

# Shape: 1 x (lag × len(properties))  →  1 x 240 with defaults
# GAuto_CIDH920105_1  GAuto_CIDH920105_2  GAuto_CIDH920105_3  GAuto_CIDH920105_4  GAuto_CIDH920105_5 ...
# 1.057               1.077               1.040               1.020               1.052 ...
```

Conjoint Triad Descriptor

Conjoint Triad (CTriad)

> **v1.6.0:** counts are normalised (f − min) / max by default, as in Shen et al. 2007 and iFeature; `normalize=False` gives raw counts.

Encodes the sequence using a 7-class reduced amino acid alphabet and computes the frequency of all consecutive three-residue combinations (triads). The 7 classes are: (1) AGV, (2) ILFP, (3) YMTS, (4) HNQW, (5) RK, (6) DE, (7) C.

```
result = protpy.conjoint_triad(protein_seq)
# Shape: 1 x 343  (7 × 7 × 7 class combinations)
# conj_triad_111  conj_triad_112  conj_triad_113  conj_triad_114  conj_triad_115 ...
```

```
# 0.409          0.773          0.500          0.136          0.273 ...
```

CTD Descriptors

> **v1.6.0:** `ctd(seq, parts="CTD")` is the CTD function (`ctd_` and the three part functions are aliases). The distribution quantile index is clamped to the first residue, as in iFeature (classes with 1-3 residues used the last residue for the 25% quantile).

CTD (Composition, Transition, Distribution) descriptors characterise the distribution of residues belonging to three physicochemical classes along the sequence. Seven physicochemical properties are supported.

Supported properties:

Property key	Description	Classes
`hydrophobicity`	Hydrophobicity	Polar / Neutral / Hydrophobic
`normalized_vdwv`	Normalised van der Waals volume	0–2.78 / 2.95–4.0 / 4.03–8.08
`polarity`	Polarity	4.9–6.2 / 8.0–9.2 / 10.4–13.0
`charge`	Charge	Positive / Neutral / Negative
`secondary_struct`	Secondary structure	Helix / Strand / Coil
`solvent_accessibility`	Solvent accessibility	Buried / Exposed / Intermediate
`polarizability`	Polarizability	0–0.108 / 0.128–0.186 / 0.219–0.409

CTD Composition

Fraction of residues in each of the three physicochemical classes.

Parameter	Type	Default	Description
`property`	str	`"hydrophobicity"`	Physicochemical property to use

```
result = protpy.ctd_composition(protein_seq)
result = protpy.ctd_composition(protein_seq, property="charge")
# hydrophobicity_CTD_C_01  hydrophobicity_CTD_C_02  hydrophobicity_CTD_C_03
# 0.279                    0.386                    0.335
```

CTD Transition

Fraction of transitions between each pair of the three physicochemical classes.

Parameter	Type	Default	Description
`property`	str	`"hydrophobicity"`	Physicochemical property to use

```
result = protpy.ctd_transition(protein_seq)
result = protpy.ctd_transition(protein_seq, property="polarity")
# hydrophobicity_CTD_T_12  hydrophobicity_CTD_T_13  hydrophobicity_CTD_T_23
# 0.181                    0.161                    0.179
```

CTD Distribution

Position of the first, 25%, 50%, 75%, and last residue of each class within the sequence (as a percentage of total length).

Parameter	Type	Default	Description
property	str	"hydrophobicity"	Physicochemical property to use

```
result = protpy.ctd_distribution(protein_seq)
result = protpy.ctd_distribution(protein_seq, property="secondary_struct")
# hydrophobicity_CTD_D_01_01  hydrophobicity_CTD_D_02_01  ...
# 0.0796                      0.557                       ...
```

CTD Combined (`ctd_`)

Calculate Composition, Transition **and** Distribution for one or all supported properties in a single call.

Parameter	Type	Default	Description
property	str	"hydrophobicity"	Property to use when all_ctd=False
all_ctd	bool	True	If True, compute CTD for all 7 supported properties

```
# All 7 properties (default)
result = protpy.ctd_(protein_seq)

# Single property
result = protpy.ctd_(protein_seq, property="charge", all_ctd=False)

# Shape: 1 x (3 + 3 + 15) per property  →  1 x 147 for all 7 properties
# CTD_C_01_hydrophobicity  CTD_C_02_hydrophobicity  CTD_C_03_hydrophobicity ...
# 0.279                    0.386                    0.335                   ...
```

Sequence Order Descriptors

Sequence order descriptors capture the effect of residue spacing along the sequence using physicochemical distance matrices. Two distance matrices are supported:

Matrix	Description
schneider-wrede	Physicochemical distance based on Schneider-Wrede scale (default)
grantham	Physicochemical distance based on Grantham's amino acid difference formula

---

Sequence Order Coupling Number — single (`sequence_order_coupling_number_`)

> **v1.6.0:** internal helper, no longer exported from `protpy` (use `protpy.sequence_order.sequence_order_coupling_number_`).

Computes the sum of squared physicochemical distances between all residue pairs separated by a gap of `d`. Returns a single float.

Parameter	Type	Default	Description
d	int	1	Gap between residue pairs
distance_matrix	str	"schneider-wrede"	Distance matrix to use

```
result = protpy.sequence_order.sequence_order_coupling_number_(protein_seq)
result = protpy.sequence_order.sequence_order_coupling_number_(protein_seq, d=5, distance_matrix="grantham")
# Returns: 401.387  (float)
```

Sequence Order Coupling Number — series (`sequence_order_coupling_number`)

> **v1.6.0:** `distance_matrix="both"` replaces `sequence_order_coupling_number_all`, and `normalize=True` divides tau_d by L − d (raw SOCN is mostly a measure of length).

Calculates SOCN values across all gaps from 1 to `lag`.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum gap value
`distance_matrix`	str	`"schneider-wrede"`	Distance matrix to use

```
# Default parameters
result = protpy.sequence_order_coupling_number(protein_seq)

# Custom lag and matrix
result = protpy.sequence_order_coupling_number(protein_seq, lag=10, distance_matrix="grantham")

# Shape: 1 x lag  →  1 x 30 with defaults
# SOCN_SW1  SOCN_SW2  SOCN_SW3  SOCN_SW4  SOCN_SW5 ...
# 401.387   409.243   376.946   393.042   396.196 ...
```

Sequence Order Coupling Number — all matrices (`sequence_order_coupling_number_all`)

> **Deprecated in v1.6.0** (removed in 2.0): use `sequence_order_coupling_number(seq, distance_matrix="both")`.

Calculates SOCN across all lags using **both** the Schneider-Wrede and Grantham matrices and concatenates the results.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum gap value

```
result = protpy.sequence_order_coupling_number_all(protein_seq)
result = protpy.sequence_order_coupling_number_all(protein_seq, lag=15)
# Shape: 1 x (2 × lag)  →  1 x 60 with defaults
# SOCN_SW1  SOCN_SW2  SOCN_SW3  SOCN_SW4  SOCN_SW5 ...
# 401.387   409.243   376.946   393.042   396.196 ...
# (SOCN_SW1-30 are followed by SOCN_Grant1-30)
```

Quasi Sequence Order (`quasi_sequence_order`)

> **v1.6.0:** the composition terms are fractions, so the 20 + lag values sum to 1 (they were 100 times too large). `distance_matrix="both"` replaces `quasi_sequence_order_all`.

Extends SOCN by combining standard amino acid composition with sequence-order coupling numbers, weighted by a factor `weight`. Captures both residue type and spatial distribution information.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum lag value
`weight`	float	`0.1`	Weighting factor for coupling terms
`distance_matrix`	str	`"schneider-wrede"`	Distance matrix to use

```
# Default parameters
result = protpy.quasi_sequence_order(protein_seq)

# Custom parameters
```

```
result = protpy.quasi_sequence_order(protein_seq, lag=10, weight=0.2, distance_matrix="grantham")

# Shape: 1 x (20 + lag)  →  1 x 50 with defaults
# QSO_SW1   QSO_SW2   QSO_SW3   QSO_SW4   QSO_SW5 ...
# 0.000057  0.000026  0.000049  0.000028  0.000056 ...
```

Quasi Sequence Order — all matrices (`quasi_sequence_order_all`)

> **Deprecated in v1.6.0** (removed in 2.0): use `quasi_sequence_order(seq, distance_matrix="both")`.

Calculates Quasi Sequence Order using **both** distance matrices and concatenates the results.

Parameter	Type	Default	Description
`lag`	int	`30`	Maximum lag value
`weight`	float	`0.1`	Weighting factor for coupling terms

```
result = protpy.quasi_sequence_order_all(protein_seq)
result = protpy.quasi_sequence_order_all(protein_seq, lag=15, weight=0.05)
# Shape: 1 x (2 × (20 + lag))  →  1 x 100 with defaults
# QSO_SW1   QSO_SW2   QSO_SW3   QSO_SW4   QSO_SW5 ...
# 0.000057  0.000026  0.000049  0.000028  0.000056 ...
# (QSO_SW1-50 are followed by QSO_Grant1-50)
```

---

v1.5.0 and v1.6.0 Descriptors

Registered descriptors added in v1.5.0 or v1.6.0 (the v1.6.0 descriptor audit merged most of the v1.5.0 additions into family functions and deprecated the rest). Each lists its formula, output shape, parameters and primary reference. Descriptors are not rounded since v1.6.0; pass `decimals=` to `all_descriptors` / `encode_dataset` to round.

Descriptor sets (`descriptor_set=` in `all_descriptors` / `encode_dataset`): **classic** (default, about 1,400 columns), **extended** (adds the low-dimensional descriptors) and **all** (adds the high-dimensional ones). Opt-in descriptors are in no set; add them with `include=[...]`.

CTD

`ctd` (CTD)

Composition, Transition and Distribution (CTD) of 3-class property alphabets for 7 properties (or the 13 of iFeature); parts= selects C, T and/or D.

```
Calculate all Composition, Transition and Distribution (CTD) features of protein sequences.
Composition is the number of amino acids of a particular property (e.g., hydrophobicity)
divided by the total number of amino acids in a protein sequence. Transition
characterizes the percent frequency with which amino acids of a particular
property is followed by amino acids of a different property. Distribution
measures the chain length within which the first, 25%, 50%, 75%, and 100% of
the amino acids of a particular property are located, respectively [1, 2].

CTD properties available are: Polarizability, Solvent Accessibility, Secondary
Structure, Charge, Polarity, Normalized VDWV, Hydrophobicity. Each property generates an
output of shape 1 x 21, 3/21 will be Composition, 3/21 will be Transition, 15/21 will be
Distribution. When Calculating all available features the generated output will be of shape
1 x 147, 21/147 will be composition, 21/147 will be transition and the remaining 105 are
distribution.
```

```
result = protpy.ctd(protein_seq)
# Shape: 1 x 147 (7 properties; 1 x 273 for ifeature13)
```

- **Columns:** `CTD_C_01_hydrophobicity`, `CTD_C_02_hydrophobicity`, `CTD_C_03_hydrophobicity`, `CTD_C_01_normalized_vdwv`, `CTD_C_02_normalized_vdwv`, `CTD_C_03_normalized_vdwv`, ...
- **Parameters:** `property`: str (default='hydrophobicity') — CTD property (used when all_ctd=False); `all_ctd`: bool (default=True) — if True, compute over all 7 properties; `property_set`: str (default='default') — 'default' (7 properties) or 'ifeature13' (13 properties, 1 x 273); `parts`: str (default='CTD') — any combination of 'C', 'T' and 'D'
- **Descriptor set:** classic, extended, all
- **Reference:** [1] Dubchak et al. (1995). PNAS, 92, 8700-8704. https://doi.org/10.1073/pnas.92.19.8700

Composition

`natural_vector` (NV)

Natural vector: count, mean position and normalised second central moment of each amino acid.

```
Calculate Natural Vector (NV) of protein sequence. The natural vector
describes the number of each amino acid in a protein sequence together with
the mean and the normalized central moment of their positions, calculated as:

n_k = number of occurrences of amino acid k

mu_k = sum of the positions of k / n_k

D2_k = sum of (position of k - mu_k)^2 / (n_k * L)

where positions start at 1, L is the length of the sequence and k is each of
the 20 canonical amino acids [26]. mu_k and D2_k are 0 when amino acid k is
not in the sequence. The n_k block is the amino acid composition times L/100;
with ``normalize=True`` n_k and mu_k are divided by L (the composition as a
fraction, and the relative mean position), so the vector no longer scales
with sequence length.
```

```
result = protpy.natural_vector(protein_seq)
# Shape: 1 x 60
```

- **Columns:** `NV_n_A`, `NV_n_C`, `NV_n_D`, `NV_n_E`, `NV_n_F`, `NV_n_G`, ...
- **Parameters:** `normalize`: bool (default=False) — divide n_k and mu_k by the sequence length
- **Descriptor set:** extended, all
- **Reference:** Yu et al. (2013). J. Theor. Biol., 318, 197-204. https://doi.org/10.1016/j.jtbi.2012.11.005

`atomic_composition` (ATC)

Percentage of C, H, N, O and S atoms over the residues' free amino-acid formulas.

```
Calculate Atomic Composition (ATC) of protein sequence. ATC describes the
percentage of carbon, hydrogen, nitrogen, oxygen and sulphur atoms in a
protein sequence, using the molecular formulas of the free amino acids,
calculated as:

ATC(e) = N_e / (N_C + N_H + N_N + N_O + N_S) * 100

where ATC(e) is the atomic composition of element e and N_e is the number of
atoms of element e summed over all residues [27]. By default the formulas of
the free amino acids are summed without removing water, matching Pfeature
[27]. With subtract_water=True, one water molecule is removed for each of the
L - 1 peptide bonds, giving the formula of the peptide (e.g. "GG" -> C4H8N2O3).
```

```
result = protpy.atomic_composition(protein_seq)
# Shape: 1 x 5
```

- **Columns:** `ATC_C`, `ATC_H`, `ATC_N`, `ATC_O`, `ATC_S`
- **Parameters:** `subtract_water`: bool (default=False) — remove (L-1) H2O
- **Descriptor set:** extended, all
- **Reference:** Pande et al. (2023). J. Comput. Biol., 30(2), 204-222 (Pfeature). https://doi.org/10.1089/cmb.2022.0241

`cksaagp` (CKSAAGP)

Composition of k-spaced amino acid group pairs over the iFeature 5-group alphabet.

```
Calculate Composition of k-Spaced Amino Acid Group Pairs (CKSAAGP) of protein
sequence. CKSAAGP is the same as CKSAAP but the amino acids are first mapped
to 5 groups: aliphatic (GAVLMI), aromatic (FYW), positive charge (KRH),
negative charge (DE) and uncharged (STCPNQ). With 25 possible group pairs
per gap this creates a 25 * (gap+1) Dimensional vector, calculated as:

CKSAAGP(G1G2, g) = N_g(G1G2) / (L - g - 1) * 100

where CKSAAGP(G1G2, g) is the composition of the group pair G1G2 with g
residues between them, N_g(G1G2) is the number of such pairs in the sequence
and L is the length of the sequence [21]. The 25 values for each gap sum to 100.
This is a permanent alias of ``cksaap(sequence, gap, alphabet="ifeature5")``.
```

```
result = protpy.cksaagp(protein_seq)
# Shape: 1 x 25·(gap+1)  (default 1 x 150)
```

- **Columns:** `CKSAAGP_aliphatic.aliphatic_gap0`, `CKSAAGP_aliphatic.aromatic_gap0`, `CKSAAGP_aliphatic.positive_gap0`, `CKSAAGP_aliphatic.negative_gap0`, `CKSAAGP_aliphatic.uncharged_gap0`, `CKSAAGP_aromatic.aliphatic_gap0`, …
- **Parameters:** `gap`: int (default=5) — maximum spacing
- **Descriptor set:** extended, all
- **Alias of:** `cksaap` (permanent literature alias)
- **Reference:** Chen et al. (2018). Bioinformatics, 34(14), 2499-2502 (iFeature). https://doi.org/10.1093/bioinformatics/bty140

Physicochemical

`charge_fractions` (FCR)

Fraction of charged residues (FCR), of positive and negative residues, net charge per residue (NCPR) and charge asymmetry (sigma).

```
Calculate the Charge Fractions (FCR) of protein sequence. The fraction of
charged residues (FCR), the fractions of positive and negative residues and
the overall charge asymmetry (sigma) describe the charge content of a protein
sequence, and are key parameters of the conformations of intrinsically
disordered proteins [1, 2]. They are calculated as:

FCR = (n+ + n-) / L

FPlus = n+ / L, FMinus = n- / L

NCPR = (n+ - n-) / L

Sigma = NCPR^2 / FCR

where n+ is the number of positive residues (K, R), n- the number of negative
residues (D, E), L is the length of the sequence and NCPR is the net charge per
residue [1]. His is treated as neutral and sigma is 0 when there are no charged
residues. (Only two of the values are independent; they are all given because
each is a standard localCIDER parameter.)
```

```
result = protpy.charge_fractions(protein_seq)
# Shape: 1 x 5
```

- **Columns:** `Charge_FCR`, `Charge_FPlus`, `Charge_FMinus`, `Charge_NCPR`, `Charge_Sigma`
- **Parameters:** sequence only
- **Descriptor set:** extended, all
- **Reference:** Das & Pappu (2013). PNAS, 110(33), 13392-13397. https://doi.org/10.1073/pnas.1304749110

`kappa` (Kappa)

Das & Pappu kappa: segregation of oppositely charged residues (0 well mixed, 1 segregated), averaged over blob sizes; NaN without charged residues.

```
Calculate the Kappa (Kappa) charge patterning parameter of protein sequence.
Kappa describes how well oppositely charged residues are mixed or segregated
along a protein sequence, which strongly affects the conformations of
intrinsically disordered proteins [2]. The sequence is split into overlapping
blobs of g residues and kappa is calculated as:

sigma_b = (f+ - f-)^2 / (f+ + f-)

delta_g = mean over all blobs of (sigma_b - sigma)^2

kappa_g = delta_g / delta_max,g

where sigma_b is the charge asymmetry of a blob, f+ and f- are its fractions of
positive (K, R) and negative (D, E) residues, sigma = NCPR^2 / FCR is the charge
asymmetry of the whole sequence and delta_max,g is the largest delta over the
most segregated arrangements of the same composition. Kappa is the mean of
kappa_g over the blob sizes (5 and 6 by default) and ranges from 0 (well
mixed) to 1 (fully segregated). Sequences with only one type of charge are
scored the same way (as in localCIDER); kappa is NaN for a sequence with no
charged residues, where it is undefined. Blobs longer than the sequence are
reduced to its length.
```

```
result = protpy.kappa(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `Kappa`
- **Parameters:** `blob_sizes`: tuple (default=(5, 6)) — blob sizes averaged into Kappa
- **Descriptor set:** extended, all
- **Reference:** Das & Pappu (2013). PNAS, 110(33), 13392-13397. https://doi.org/10.1073/pnas.1304749110

`sequence_charge_decoration` (SCD)

Sequence charge decoration: (1/L) sum_{m>n} q_m q_n sqrt(m-n).

```
Calculate the Sequence Charge Decoration (SCD) of protein sequence. SCD
describes the patterning of charges along a protein sequence, weighting each
pair of charges by the square root of their separation [3]. It is calculated as:

SCD = (1/L) * sum_{m=2}^{L} sum_{n=1}^{m-1} q_m * q_n * (m - n)^(1/2)

where q_m and q_n are the charges of the residues at positions m and n (+1 for
K and R, -1 for D and E, 0 otherwise) and L is the length of the sequence. More
negative values indicate more segregated opposite charges and uncharged
sequences have an SCD of 0.
```

```
result = protpy.sequence_charge_decoration(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `Charge_SCD`
- **Parameters:** sequence only
- **Descriptor set:** extended, all
- **Reference:** Sawle & Ghosh (2015). J. Chem. Phys., 143(8), 085101. https://doi.org/10.1063/1.4929391

`hydrophobicity_profile` (HP)

Sliding-window profile of any single-value scale (hydropathy, flexibility, Parker, Janin, TOP-IDP, ...) summarised by mean, max, min and std; optional threshold fraction and segment (e.g. transmembrane) features.

```
Calculate the Hydrophobicity Profile (HP) of protein sequence. Each amino acid
in the sequence is replaced by its value on a hydrophobicity scale (or any
other single-value scale) and the values are averaged over a sliding window,
giving a profile along the sequence [5, 6]. The profile is summarised by its
mean, maximum, minimum and standard deviation, and optionally the fraction of
windows above a threshold, e.g. 1.6 for transmembrane segments with the
Kyte-Doolittle scale [5]. The window value is calculated as:

HP(i) = sum_j w_j * H(s_i+j) / sum_j w_j

where H is the scale and w_j are the window weights. With edge=1.0 all
weights are 1 (a plain mean); with edge < 1 the weights fall linearly from 1 at
the centre of the window to edge at its ends, as in ExPASy ProtScale. Windows
longer than the sequence use the whole sequence.
```

```
With ``segments=True`` the windows above ``threshold`` are merged into
segments (the residues covered by at least one such window), and the number of
segments, the fraction of residues inside a segment and the positive-inside
bias (K + R in the 15 residues before the first segment minus K + R in the 15
residues after it, 0 without a segment) [14] are added. With the Kyte-Doolittle
scale, window=19 and threshold=1.6 these are hydropathy-based transmembrane
segments; this is a heuristic, not a topology predictor (signal peptides are
also counted, and multi-pass proteins are often under-counted).

The scale can be any protpy scale (e.g. "vihinen_flexibility" with window=9
and edge=0.25 gives the Vihinen flexibility profile [13], "parker" or "janin"
with window=7 the Parker and Janin profiles) or an AAindex1 accession number.
```

```
result = protpy.hydrophobicity_profile(protein_seq)
# Shape: 1 x 4 (5 with threshold, 8 with segments)
```

- **Columns:** `HP_kyte_doolittle_w9_mean`, `HP_kyte_doolittle_w9_max`, `HP_kyte_doolittle_w9_min`, `HP_kyte_doolittle_w9_std`
- **Parameters:** `scale`: str (default='kyte_doolittle') — any single-value protpy scale or AAindex code; `window`: int (default=9) — window length; `edge`: float (default=1.0) — relative end weight; `threshold`: float | None (default=None) — adds fraction of windows above threshold; `segments`: bool (default=False) — add segment count, fraction and positive-inside bias (needs threshold)
- **Descriptor set:** extended, all
- **Reference:** Kyte & Doolittle (1982). J. Mol. Biol., 157(1), 105-132. https://doi.org/10.1016/0022-2836(82)90515-0

`thermostability_indices` (Thermo)

IVYWREL index and charged-versus-polar (CvP) bias, composition-based thermostability indicators.

```
Calculate the Thermostability Indices of protein sequence: the IVYWREL index
and the charged versus polar (CvP) bias, composition-based indicators of
protein thermostability [11, 12]. They are calculated as:

IVYWREL = N(IVYWREL) / L

CvP = %(D + E + K + R) - %(N + Q + S + T)

where N(IVYWREL) is the number of I, V, Y, W, R, E and L residues, %(X) is the
percentage of residues of type X and L is the length of the sequence. IVYWREL
was derived as a proteome-wide predictor of optimal growth temperature and
does not reliably rank orthologous proteins; CvP bias was validated on
hyperthermophile proteomes. Both are fixed linear maps of the amino acid
composition.
```

```
result = protpy.thermostability_indices(protein_seq)
# Shape: 1 x 2
```

- **Columns:** `IVYWREL`, `CvP_Bias`
- **Parameters:** sequence only
- **Descriptor set:** extended, all
- **Reference:** Zeldovich, Berezovsky & Shakhnovich (2007). PLoS Comput. Biol., 3(1), e5. https://doi.org/10.1371/journal.pcbi.0030005

Complexity

`wootton_federhen_complexity` (WF)

Wootton-Federhen local complexity: mean and minimum windowed entropy K2 and the fraction of residues in low-complexity windows (trigger step only, not masked regions).

```
Calculate the Wootton-Federhen Complexity (WF) of protein sequence. The
Wootton-Federhen complexity describes the local compositional complexity of a
protein sequence over a sliding window [1, 2]. For each window of W residues
it is calculated as:

K2 = -sum_i (n_i / W) * log2(n_i / W)
```

```
where K2 is the Shannon entropy of the window in bits and n_i is the number of
residues of amino acid i in the window. The low-complexity fraction is the
fraction of residues covered by at least one window with K2 <= k2_trigger.
This is only the trigger step of a low-complexity search: segments are not
extended or merged, so the output is a descriptor and not the masked regions
a low-complexity masking program reports. A window longer than the sequence
uses the whole sequence. (The K1 complexity state was dropped in v1.6.0; its
correlation with K2 was >= 0.996.)
```

```
result = protpy.wootton_federhen_complexity(protein_seq)
# Shape: 1 x 3
```

- **Columns:** `WF_K2_mean`, `WF_K2_min`, `WF_LowComplexity_frac`
- **Parameters:** `window`: int (default=12) — window length; `k2_trigger`: float (default=2.2) — K2 trigger (bits)
- **Descriptor set:** extended, all
- **Reference:** Wootton & Federhen (1993). Comput. Chem., 17(2), 149-163. https://doi.org/10.1016/0097-8485(93)85006-X

`lempel_ziv_complexity` (LZ)

Normalised Lempel-Ziv (LZ76) complexity c(n) log_20(n) / n.

```
Calculate the Lempel-Ziv Complexity (LZ) of protein sequence. The Lempel-Ziv
(LZ76) complexity is the number of new patterns, c(n), found when a sequence is
read from left to right, each pattern being the shortest substring that cannot
be copied from the preceding part of the sequence [4, 5]. As c(n) grows with the
length of the sequence (r = 0.999 with length on real proteins), only the
normalised value is returned:

LZ_Normalized = c(n) * log_20(n) / n

where n is the length of the sequence and 20 is the size of the amino acid
alphabet. The normalised value is 0 for a single residue.
```

```
result = protpy.lempel_ziv_complexity(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `LZ_Normalized`
- **Parameters:** sequence only
- **Descriptor set:** extended, all
- **Reference:** Lempel & Ziv (1976). IEEE Trans. Inf. Theory, 22(1), 75-81. https://doi.org/10.1109/TIT.1976.1055501

Spectral

`spectral_descriptors` (Spectral)

Binned FFT power spectrum (default EIIP) or DCT coefficients 1..n (default Kyte-Doolittle) of a property signal.

```
Calculate Spectral descriptors of protein sequence. Each amino acid in the
sequence is replaced by its value on a property scale and the mean-centred
signal x is transformed into a fixed number of features.

transform="fft": the power spectrum of the signal [1],

P(f) = abs(FFT(x)(f))^2 / L

where f is the frequency in cycles per residue (0 to 0.5) and L is the length
of the sequence. The frequency range 0-0.5 is split into n equal-width bins and
the mean power in each bin is returned; bins containing no frequencies are 0.
The default scale is the electron-ion interaction potential (EIIP) of the
resonant recognition model [1].

transform="dct": coefficients 1..n of the orthonormal type-II discrete cosine
transform of the signal, which compresses it into a fixed number of values [2]:

X_k = sqrt(2/L) * sum_{n=0}^{L-1} x_n * cos(pi * k * (2n + 1) / (2L))

X_0 is always 0 for a mean-centred signal, so it is not returned. The default
```

```
scale is the Kyte-Doolittle hydropathy.
```

```
result = protpy.spectral_descriptors(protein_seq)
# Shape: 1 x n  (default 1 x 10)
```

- **Columns:** FFT_eiip_bin1, FFT_eiip_bin2, FFT_eiip_bin3, FFT_eiip_bin4, FFT_eiip_bin5, FFT_eiip_bin6, ...
- **Parameters:** transform: str (default='fft') — 'fft' (binned power spectrum) or 'dct' (coefficients 1..n); scale: str | dict | None (default=None) — single-value scale; None uses 'eiip' (FFT) or 'kyte_doolittle' (DCT); n: int (default=10) — number of FFT bins or DCT coefficients
- **Descriptor set:** extended, all
- **Reference:** Cosic (1994). IEEE Trans. Biomed. Eng., 41(12), 1101-1114. https://doi.org/10.1109/10.335859

Structural

aggrescan (AGG)

AGGRESCAN aggregation hot-spot descriptors from the a3v propensity profile (hot spots: >=5 residues above -0.02, no Pro).

```
Calculate AGGRESCAN (AGG) aggregation descriptors of protein sequence.
AGGRESCAN predicts aggregation "hot spots", short stretches that promote
protein aggregation, from experimental aggregation propensities (a3v) of the
20 amino acids [1]. The a3v values are averaged over a sliding window, giving
the aggregation profile (a4v), with each window value assigned to its central
residue and the off-centre N- and C-terminal residues given the first and last
window values. A hot spot is a run of 5 or more residues with an a4v above the
hot-spot threshold (HST) of -0.02 and containing no proline. Areas are
calculated by trapezoidal integration of the profile, with HST as the zero axis.

By default the 5 length-normalised outputs are returned: the mean a3v of the
sequence (a3vSA), the number of hot spots per 100 residues (NnHS), the area of
the profile above HST per residue (AATr), the total hot-spot area per residue
(THSAr) and the mean a4v in the hot spots (a4vAHS, 0 if there are none). The
other AGGRESCAN outputs are length-scaled copies of these, and are returned
with ``full=True``: the number of hot spots (nHS), the area above HST (AAT),
the total hot-spot area (THSA), the total area of the profile (TA), the sum of
a4v (a4vSS) and per 100 residues (Na4vSS), and the largest single hot-spot area
(HSA_max).

The AGGRESCAN server chooses a window of 5, 7, 9 or 11 residues from the
sequence length (<= 75, <= 175, <= 300, > 300 residues). This rule is not in
the published supplementary material; it is given by later descriptions of
the server and the psico reimplementation, and is available as
``window="psico"``. The documented default is 5. The server's specific values
for the two extreme residues are not published, so they are treated like the
other off-centre residues.
```

```
result = protpy.aggrescan(protein_seq)
# Shape: 1 x 5 (1 x 12 with full=True)
```

- **Columns:** AGG_a3vSA, AGG_NnHS, AGG_AATr, AGG_THSAr, AGG_a4vAHS
- **Parameters:** window: int | str (default=5) — odd window length, or 'psico' for the server's length rule (5/7/9/11); full: bool (default=False) — return all 12 AGGRESCAN outputs instead of the 5 normalised ones
- **Descriptor set:** extended, all
- **Reference:** Conchillo-Solé et al. (2007). BMC Bioinformatics, 8, 65. https://doi.org/10.1186/1471-2105-8-65

---

charge_hydropathy (CH)

Uversky charge-hydropathy score HQ = 2.785 - abs() - 1.151 of the whole sequence, or its FoldIndex sliding-window profile (window=51).

```
Calculate the Charge-Hydropathy (CH) score of protein sequence. Uversky et al.
showed that natively unfolded proteins can be separated from folded proteins
by their mean net charge and mean hydropathy [2], with the boundary
<R> = 2.785 <H> - 1.151 (as restated in [15]). The score is the distance from
that boundary:

HQ = 2.785 <H> - <R> - 1.151
```

```
<H> = mean of (KD + 4.5) / 9,  <R> = abs(n+ - n-) / n

where KD is the Kyte-Doolittle hydropathy, n+ and n- the numbers of positive
(K, R) and negative (D, E) residues and n the number of residues considered.
Negative scores are on the unfolded side of the boundary.

With window=None the score of the whole sequence is returned (the Uversky
plot). With a window, the score of the window centred on each residue is
calculated, as in FoldIndex (window=51) [3], and the profile is summarised by
its mean, maximum, minimum and standard deviation. This reports the score, not
a disorder prediction: the FoldIndex window and its residue assignment could
not be checked against [3].
```

```
result = protpy.charge_hydropathy(protein_seq)
# Shape: 1 x 1 (window=None) or 1 x 4
```

- **Columns:** `CH_HQ`
- **Parameters:** `window`: int | None (default=None) — None scores the whole sequence; e.g. 51 gives the FoldIndex profile
- **Descriptor set:** extended, all
- **Reference:** Uversky, Gillespie & Fink (2000). Proteins, 41(3), 415-427. https://doi.org/10.1002/1097-0134(20001115)41:3<415::AID-PROT130>3.0.CO;2-7

`plaac` (PLAAC)

PLAAC prion-like domain scores: max window LLR, HMM core score, PrD length and presence.

```
Calculate the Prion-Like Amino Acid Composition (PLAAC) descriptors of protein
sequence. PLAAC identifies prion-like domains, whose composition resembles
that of yeast prion domains, using the amino acid frequencies of prion domains
and of the S. cerevisiae proteome [11]. Each residue has a log-likelihood ratio:

LLR(s_i) = ln(f_prion(s_i) / f_background(s_i))

The 4 output values are: the maximum sum of LLR over any window of core_length
residues (PLAAC_LLR); the maximum sum of LLR over a window of core_length
residues lying entirely inside a prion-like region of the two-state hidden
Markov model Viterbi parse (PLAAC_COREscore); the length of the prion-like
region containing that core (PLAAC_PrD_length); and whether the sequence has a
prion-like region of at least core_length residues (PLAAC_has_PrD). As in
PLAAC, PLAAC_LLR and PLAAC_COREscore are NaN when they are undefined (a
sequence shorter than core_length, or no prion-like region of that length).
The parameters are ported from the PLAAC source code (MIT License), using the
S. cerevisiae background (alpha = 1).
```

```
result = protpy.plaac(protein_seq)
# Shape: 1 x 4
```

- **Columns:** `PLAAC_LLR`, `PLAAC_COREscore`, `PLAAC_PrD_length`, `PLAAC_has_PrD`
- **Parameters:** `core_length`: int (default=60) — minimal core length; `fill_value`: float (default=nan) — value of PLAAC_LLR / PLAAC_COREscore where undefined
- **Descriptor set:** extended, all
- **Reference:** Lancaster et al. (2014). Bioinformatics, 30(17), 2501-2502. https://doi.org/10.1093/bioinformatics/btu310

Composition

`cksaap` (CKSAAP)

Composition of k-spaced amino acid pairs: CKSAAP(XY,g) = N_g(XY) / (L-g-1) x 100 for g = 0..gap; alphabet= groups the residues (CKSAAGP, distance pair).

```
Calculate Composition of k-Spaced Amino Acid Pairs (CKSAAP) of protein
sequence. CKSAAP describes the frequency of each amino acid pair separated
by g residues within a protein sequence, for each gap g in the range
0..gap. With 400 possible amino acid pairs per gap this creates a
400 * (gap+1) Dimensional vector, calculated as:

CKSAAP(XY, g) = N_g(XY) / (L - g - 1) * 100
```

```
where CKSAAP(XY, g) is the composition of the amino acid pair XY with g
residues between X and Y, N_g(XY) is the number of such pairs in the sequence
and L is the length of the sequence [20]. A gap of 0 is the same as the
dipeptide composition. A gap with no pairs (L - g - 1 <= 0) gives values
of 0.

With ``alphabet`` the amino acids are first mapped to the groups of a reduced
alphabet (see ``kmer_composition``), giving G^2 group pairs per gap.
``alphabet="ifeature5"`` is the composition of k-spaced amino acid group pairs
(CKSAAGP) of iFeature [21], also available as ``cksaagp()``.
```

```
result = protpy.cksaap(protein_seq)
# Shape: 1 x 400·(gap+1)  (default 1 x 2400), or G²·(gap+1) with an alphabet
```

- **Columns:** `CKSAAP_AA_gap0`, `CKSAAP_AC_gap0`, `CKSAAP_AD_gap0`, `CKSAAP_AE_gap0`, `CKSAAP_AF_gap0`, `CKSAAP_AG_gap0`, …
- **Parameters:** `gap`: int (default=5) — maximum spacing; `alphabet`: int | str | list | None (default=None) — reduced alphabet (see kmer_composition)
- **Descriptor set:** all
- **Reference:** Chen, Kurgan & Ruan (2007). BMC Struct. Biol., 7, 25. https://doi.org/10.1186/1472-6807-7-25

`asdc` (ASDC)

Adaptive skip dipeptide composition: composition of all ordered pairs (s_i, s_j), i < j.

```
Calculate Adaptive Skip Dipeptide Composition (ASDC) of protein sequence.
ASDC is a modified dipeptide composition that counts every ordered pair of
amino acids (s_i, s_j), where i < j, at all gaps along the sequence, rather
than just adjacent pairs, calculated as:

ASDC(XY) = sum_g N_g(XY) / sum_g sum_XY N_g(XY) * 100

where ASDC(XY) is the composition of the amino acid pair XY, N_g(XY) is the
number of XY pairs separated by g residues, and the denominator is the total
number of ordered pairs in the sequence, L(L-1)/2, where L is the length of
the sequence [23]. The 400 values sum to 100.
```

```
result = protpy.asdc(protein_seq)
# Shape: 1 x 400
```

- **Columns:** `ASDC_AA`, `ASDC_AC`, `ASDC_AD`, `ASDC_AE`, `ASDC_AF`, `ASDC_AG`, …
- **Parameters:** sequence only
- **Descriptor set:** all
- **Reference:** Wei et al. (2018). Bioinformatics, 34(23), 4007-4016 (ACPred-FL). https://doi.org/10.1093/bioinformatics/bty451

`distance_pair` (DP)

PseAAC of distance-pairs over a reduced cluster profile (cp13/cp14/cp19/cp20).

```
Calculate Distance Pair (DP) descriptor of protein sequence. The descriptor
incorporates amino acid distance pairs and a reduced amino acid alphabet into
the general pseudo amino acid composition, as used in iDNA-Prot|dis [24, 25].
The amino acids are first mapped to the clusters of a cluster profile cp(n),
and the descriptor is the composition of the clusters followed by the
composition of cluster pairs at each distance d = 1..rng:

DP(c) = N(c) / L * 100

DP(c1c2, d) = N_d(c1c2) / (L - d) * 100

where N(c) is the number of residues in cluster c, N_d(c1c2) is the number of
residue pairs (s_i, s_i+d) whose residues are in clusters c1 and c2, and L is
the length of the sequence. A distance of 1 is the nearest-neighbour pair.
The available cluster profiles are cp13, cp14, cp19 and cp20 (no reduction).
The published cp(14) lists E twice; the extra E in the first cluster is
dropped, as it is the only reading that gives 14 clusters.
```

```
result = protpy.distance_pair(protein_seq)
# Shape: 1 x (G + G²·rng)  (default 1 x 602)
```

- **Columns:** DP_IMV, DP_L, DP_F, DP_WY, DP_G, DP_P, …
- **Parameters:** rng: int (default=3) — maximum pair distance; alphabet: str (default='cp14') — cluster profile
- **Descriptor set:** all
- **Alias of:** cksaap (permanent literature alias)
- **Reference:** Liu et al. (2014). PLoS ONE, 9(9), e106691 (iDNA-Prot|dis). https://doi.org/10.1371/journal.pone.0106691

Scales

auto_cross_covariance (ACC)

Auto/cross-covariance of an amino-acid scale: sum z_j(i) z_k(i+l) / (L-l).

```
Calculate Auto Cross Covariance (ACC) of protein sequence. ACC transforms the
amino acid scale values of a sequence into a fixed-length vector that
describes the covariance between the scale dimensions of residues separated
by l positions, for each lag l in the range 1..lag [10, 11]. It is calculated as:

ACC_jk(l) = sum_{i=1}^{L-l} z_j(s_i) * z_k(s_i+l) / (L - l)

where ACC_jk(l) is the covariance between dimensions j and k at lag l, z_j(s_i)
is the value of dimension j for the amino acid at position i and L is the length
of the sequence. When j = k this is the auto covariance and when j != k the cross
covariance. The scale can be one of the factor-scale families, which are used as
published, or one or more AAindex accession numbers, whose values are first
standardised over the 20 amino acids.
```

```
result = protpy.auto_cross_covariance(protein_seq)
# Shape: 1 x d²·lag  (default 1 x 125)
```

- **Columns:** ACC_1_1_lag1, ACC_1_2_lag1, ACC_1_3_lag1, ACC_1_4_lag1, ACC_1_5_lag1, ACC_2_1_lag1, …
- **Parameters:** scale: str | list[str] (default='z_scales_5') — factor scale or AAindex code(s); lag: int (default=5) — maximum lag; mode: str (default='acc') — 'acc', 'ac' or 'cc'
- **Descriptor set:** all
- **Reference:** Wold et al. (1993). Anal. Chim. Acta, 277(2), 239-253. https://doi.org/10.1016/0003-2670(93)80437-P

Composition

dde (DDE)

Dipeptide deviation from expected mean: (Dc - Tm) / sqrt(Tv), with Tm from codon counts.

```
Calculate Dipeptide Deviation from Expected Mean (DDE) of protein sequence.
DDE compares the observed dipeptide composition of a protein sequence with
the composition expected from the number of codons that code for each amino
acid [22]. This is a permanent alias of
``dipeptide_composition(sequence, normalize="dde")``, which documents the
formula. Unlike the other composition descriptors, DDE values are not
percentages.
```

```
result = protpy.dde(protein_seq)
# Shape: 1 x 400
```

- **Columns:** DDE_AA, DDE_AC, DDE_AD, DDE_AE, DDE_AF, DDE_AG, …
- **Parameters:** sequence only
- **Descriptor set:** opt-in (include=[...])
- **Alias of:** dipeptide_composition (permanent literature alias)
- **Reference:** Saravanan & Gautham (2015). OMICS, 19(10), 648-658. https://doi.org/10.1089/omi.2015.0095

Autocorrelation

`autocorrelation` (Auto)

Moreau-Broto, Moran or Geary autocorrelation of standardised AAindex properties for lags 1..lag.

```
Calculate an Autocorrelation descriptor of protein sequence. Autocorrelation
descriptors are topological descriptors that describe the level of correlation
between residues separated by d positions in terms of a physicochemical
property [1, 2]. For a property P (by default standardised over the 20 amino
acids) and each lag d = 1..lag they are calculated as:

Moreau-Broto: MB(d) = sum_{i=1}^{L-d} P_i * P_{i+d} / (L - d)

Moran:        I(d)  = [sum_{i=1}^{L-d} (P_i - P') * (P_{i+d} - P') / (L - d)]
                      / [sum_{i=1}^{L} (P_i - P')^2 / L]

Geary:        C(d)  = [sum_{i=1}^{L-d} (P_i - P_{i+d})^2 / (2 (L - d))]
                      / [sum_{i=1}^{L} (P_i - P')^2 / (L - 1)]

where P_i is the property value of the residue at position i, P' is the mean
of P over the sequence and L is the length of the sequence. Moran and Geary
are 0 for a sequence whose property values do not vary. With the default 8
AAindex properties and lag=30, 240 features are generated. Ong et al. found no
clearly preferred method among the three [2]; Moran and Geary are strongly
(negatively) correlated at the same property and lag, and Moreau-Broto equals
``auto_cross_covariance(mode="ac")`` on the same properties. The method names
are also available as ``moreaubroto_autocorrelation``,
``moran_autocorrelation`` and ``geary_autocorrelation``.
```

```
result = protpy.autocorrelation(protein_seq)
# Shape: 1 x (n_properties × lag)  (default 1 x 240)
```

- **Columns:** `GAuto_CIDH920105_1`, `GAuto_CIDH920105_2`, `GAuto_CIDH920105_3`, `GAuto_CIDH920105_4`, `GAuto_CIDH920105_5`, `GAuto_CIDH920105_6`, ...
- **Parameters:** `method`: str (default='geary') — 'moreaubroto', 'moran' or 'geary'; `lag`: int (default=30) — max lag between residue pairs; `properties`: list[str] (default=8 AAIndex codes) — AAIndex1 accession numbers; `normalize`: bool (default=True) — standardise property values (z-score) before calculation
- **Descriptor set:** opt-in (include=[...])
- **Reference:** [2] Ong, S. A. K. et al. (2007). BMC Bioinformatics, 8, 300. https://doi.org/10.1186/1471-2105-8-300

Scales

`aa_scale_descriptors` (Scale)

Mean (or per-position) values of an amino acid factor scale: z-scales, VHSE, Atchley, T-scales, ST-scales, ProtFP, FASGAI, BLOSUM indices or MS-WHIM.

```
Calculate an amino acid factor-scale descriptor of protein sequence. Factor
scales describe each amino acid with a small number of values (dimensions)
derived from many physicochemical properties, typically by principal component
or factor analysis [1-9]. Each amino acid in the sequence is replaced by its
d-dimensional scale vector and, with mode="mean", the mean of each dimension is
taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for dimension j (j = 1,2,..,d), z_j(s_i) is
the value of dimension j for the amino acid at position i and L is the length
of the sequence. With domains > 1 the sequence is split into that many
near-equal contiguous segments and the means of each segment are concatenated.

The sequence mean is an exact linear function of the amino acid composition,
so it adds no information to it; the scales were benchmarked as per-residue
encodings of binding sites or mutated positions [6]. ``mode="position"``
returns that encoding: the d values of every position, concatenated (1 x L*d),
which needs aligned sequences of equal length (e.g. a mutant library).

The z_scales_5, vhse, st_scales and fasgai tables are experimental: some of
their transcribed values could not be checked against the primary papers, and
using them raises a UserWarning (see protpy/data/aa_scales.json). The ST-scales
```

```
also ranked last of 13 sets in [6].
```

```
result = protpy.aa_scale_descriptors(protein_seq, scale='atchley_factors')
# Shape: 1 x d·domains (mode='mean') or 1 x L·d (mode='position')
```

- **Columns:** `AF_1`, `AF_2`, `AF_3`, `AF_4`, `AF_5`
- **Parameters:** `scale`: str — factor scale: 'z_scales_5', 'vhse', 'atchley_factors', 't_scales', 'st_scales', 'protfp', 'fasgai', 'blosum_indices' or 'ms_whim'; `domains`: int (default=1) — number of near-equal segments; `mode`: str (default='mean') — 'mean' or 'position' (per-residue values, aligned sequences)
- **Descriptor set:** opt-in (include=[...])
- **Reference:** van Westen et al. (2013). J. Cheminform., 5, 41. https://doi.org/10.1186/1758-2946-5-41

Deprecated in v1.6.0

These functions still work but emit a `DeprecationWarning`; they will be removed in protpy 2.0. They are no longer registered, so they cannot be used in `include`/`params`.

Function	Use instead	Reason
`hydrophobic_polar_charged_composition`	`kmer_composition(seq, k=1, alphabet=3)`	linear in AAC; left P out of every class
`secondary_structure_propensity`	`gravy(seq, scale="CHOP780201")`	linear in AAC; tables match no AAindex record
`reduced_alphabet_composition`	`kmer_composition(seq, k, alphabet=...)`	merged
`amino_acid_pair_composition`	`dipeptide_composition`	exact duplicate of DPC
`aggregation_propensity`	`aggrescan`	unpublished heuristic
`correlation_function, amphiphilic_correllation_function`	—	no call sites
`sequence_order_coupling_number_all`	`sequence_order_coupling_number(seq, distance_matrix="both")`	merged
`quasi_sequence_order_all`	`quasi_sequence_order(seq, distance_matrix="both")`	merged
`z_scales_5, vhse, atchley_factors, t_scales, protfp, fasgai, blosum_indices, ms_whim`	`aa_scale_descriptors(seq, "<name>")`	merged; sequence means are linear in AAC
`st_scales`	`aa_scale_descriptors(seq, "st_scales")`	ranked last of 13 sets; unresolved sign flip
`physicochemical_distance_transformation`	`autocorrelation(seq, method="geary")`	PDT = 2 s² Geary exactly
`ncpr`	`charge_fractions (Charge_NCPR)`	merged
`charge_density`	`charge_distribution(seq, normalize="mass")`	merged
`hydrophobic_ratio`	`kmer_composition(seq, k=1, alphabet=3)`	linear in AAC
`half_life`	—	first residue only; nearly constant
`ivywrel_index, cvp_bias`	`thermostability_indices`	merged
`flexibility`	`hydrophobicity_profile(seq, scale="vihinen_flexibility", window=9, edge=0.25)`	merged (identical values)
`linguistic_complexity`	`lempel_ziv_complexity`	O(L²), saturates near 1, formula unverified
`fft_descriptors, dct_descriptors`	`spectral_descriptors(seq, transform="fft"/"dct")`	merged; DCT_0 dropped
`uversky_charge_hydropathy`	`charge_hydropathy(seq)`	merged
`fold_index`	`charge_hydropathy(seq, window=51)`	merged; residue assignment fixed
`disorder_propensity`	`hydrophobicity_profile(seq, scale="top_idp")`	linear in AAC

Function	Use instead	Reason
`transmembrane_features`	`hydrophobicity_profile(seq, window=19, threshold=1.6, segments=True)`	merged
`functional_site_counts`	`motif_composition(seq, motifs=[...], per_100=True)`	merged
`signal_peptide_features`	—	not a signal-peptide predictor
`antigenicity_surface`	`hydrophobicity_profile(seq, scale="parker", window=7)`	exact duplicate
`cysteine_features`	`amino_acid_composition`, `motif_composition`	duplicates AAC and a motif

The helpers `sequence_order_coupling_number_`, `str_to_num`, `normalize_property`, `sequence_order_correlation_factor` and `amphiphilic_sequence_order_correlation_factor` are no longer exported from the `protpy` namespace (they remain in their modules); `sequence_order_coupling_number_`, `sequence_order_correlation_factor` and `amphiphilic_sequence_order_correlation_factor` are no longer used by protpy and are deprecated.

Deprecated descriptors: reference

Each deprecated v1.5.0 function is documented below as it behaves in v1.6.0, with its replacement. Calling one raises a `DeprecationWarning`; all are removed in protpy 2.0. (The deprecated v1.4 functions keep their entries in the sections above.)

`z_scales_5` (deprecated)

```
Calculate Sandberg 5 z-scales (Z5) of protein sequence. The z-scales are the
principal components of a set of physicochemical properties measured for 87
coded and non-coded amino acids [1]. z1 mainly describes lipophilicity, z2
steric bulk and polarizability, z3 polarity and charge, and z4 and z5
electronegativity, heat of formation, electrophilicity and hardness.

Each amino acid in the sequence is replaced by its 5 z-scales and the mean of
each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "z_scales_5")``.
```

```
result = protpy.z_scales_5(protein_seq)
# Shape: 1 x 5
```

- **Columns:** `Z5_1`, `Z5_2`, `Z5_3`, `Z5_4`, `Z5_5`
- **Parameters:** `domains`
- **Use instead:** `protpy.aa_scale_descriptors(seq, "z_scales_5")`

`vhse` (deprecated)

```
Calculate Vectors of Hydrophobic, Steric and Electronic properties (VHSE) of
protein sequence. The VHSE scales are principal components score vectors derived
from physicochemical variables of the 20 coded amino acids [2]. VHSE1-2 describe
hydrophobic properties, VHSE3-4 steric properties and VHSE5-8 electronic
properties.

Each amino acid in the sequence is replaced by its 8 VHSE scales and the mean of
each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "vhse")``.
```

```
result = protpy.vhse(protein_seq)
# Shape: 1 x 8
```

- **Columns:** VHSE_1, VHSE_2, VHSE_3, VHSE_4, VHSE_5, VHSE_6, ...
- **Parameters:** domains
- **Use instead:** `protpy.aa_scale_descriptors(seq, "vhse")`

`atchley_factors` (deprecated)

```
Calculate Atchley Factors (AF) of protein sequence. The Atchley factors are 5
factors obtained from a factor analysis of 54 amino acid attributes [3]. Factor
I mainly reflects polarity, accessibility and hydrophobicity, factor II
propensity for secondary structure, factor III molecular size, factor IV
relative amino acid composition and factor V electrostatic charge.

Each amino acid in the sequence is replaced by its 5 Atchley factors and the
mean of each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "atchley_factors")``.
```

```
result = protpy.atchley_factors(protein_seq)
# Shape: 1 x 5
```

- **Columns:** AF_1, AF_2, AF_3, AF_4, AF_5
- **Parameters:** domains
- **Use instead:** `protpy.aa_scale_descriptors(seq, "atchley_factors")`

`t_scales` (deprecated)

```
Calculate Topological Scales (T-scales) of protein sequence. The T-scales are
principal components of topological descriptors of amino acids, describing their
structural and topological characteristics [4].

Each amino acid in the sequence is replaced by its 5 T-scales and the mean of
each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "t_scales")``.
```

```
result = protpy.t_scales(protein_seq)
# Shape: 1 x 5
```

- **Columns:** T_1, T_2, T_3, T_4, T_5
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "t_scales")

---

st_scales (deprecated)

```
Calculate Structural Topology Scales (ST-scales) of protein sequence. The ST-
scales are principal components of 3D structural and topological properties of
amino acids [5].

Each amino acid in the sequence is replaced by its 8 ST-scales and the mean of
each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "st_scales")``. The ST-scales ranked last of 13
descriptor sets in [6] and their table has an unresolved sign flip.
```

```
result = protpy.st_scales(protein_seq)
# Shape: 1 x 8
```

- **Columns:** ST_1, ST_2, ST_3, ST_4, ST_5, ST_6, ...
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "st_scales")

---

protfp (deprecated)

```
Calculate ProtFP (ProtFP) of protein sequence. ProtFP is a set of 8 principal
components calculated from a selection of physicochemical properties in the
AAindex database [6].

Each amino acid in the sequence is replaced by its 8 ProtFP components and the
mean of each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "protfp")``.
```

```
result = protpy.protfp(protein_seq)
# Shape: 1 x 8
```

- **Columns:** ProtFP_1, ProtFP_2, ProtFP_3, ProtFP_4, ProtFP_5, ProtFP_6, ...
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "protfp")

`fasgai` (deprecated)

```
Calculate Factor Analysis Scales of Generalized Amino Acid Information (FASGAI)
of protein sequence. The FASGAI scales are 6 factors obtained from a factor
analysis of amino acid properties, describing hydrophobicity, alpha and turn
propensities, bulky properties, compositional characteristics, local flexibility
and electronic properties [7].

Each amino acid in the sequence is replaced by its 6 FASGAI factors and the mean
of each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "fasgai")``.
```

```
result = protpy.fasgai(protein_seq)
# Shape: 1 x 6
```

- **Columns:** FASGAI_1, FASGAI_2, FASGAI_3, FASGAI_4, FASGAI_5, FASGAI_6
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "fasgai")

`blosum_indices` (deprecated)

```
Calculate BLOSUM Indices (BLOSUM) of protein sequence. The BLOSUM indices are 10
factors derived from the BLOSUM62 substitution matrix by principal component
analysis and VARIMAX rotation, so they describe the evolutionary similarity of
amino acids [8].

Each amino acid in the sequence is replaced by its 10 BLOSUM indices and the
mean of each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "blosum_indices")``.
```

```
result = protpy.blosum_indices(protein_seq)
# Shape: 1 x 10
```

- **Columns:** BLOSUM_1, BLOSUM_2, BLOSUM_3, BLOSUM_4, BLOSUM_5, BLOSUM_6, ...
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "blosum_indices")

`ms_whim` (deprecated)

```
Calculate Molecular Surface Weighted Holistic Invariant Molecular Scores (MS-
WHIM) of protein sequence. The MS-WHIM scores are 3 principal components of 3D
descriptors calculated from the molecular electrostatic potential on the surface
of each amino acid [9].

Each amino acid in the sequence is replaced by its 3 MS-WHIM scores and the mean
of each is taken over the sequence, calculated as:

S_j = (1/L) * sum_i z_j(s_i)

where S_j is the descriptor value for scale j, z_j(s_i) is the value of scale
j for the amino acid at position i and L is the length of the sequence. With
domains > 1 the sequence is split into that many near-equal segments and the
means of each segment are concatenated.

Deprecated since v1.6.0, removed in 2.0: use
``aa_scale_descriptors(sequence, "ms_whim")``.
```

```
result = protpy.ms_whim(protein_seq)
# Shape: 1 x 3
```

- **Columns:** MSWHIM_1, MSWHIM_2, MSWHIM_3
- **Parameters:** domains
- **Use instead:** protpy.aa_scale_descriptors(seq, "ms_whim")

---

physicochemical_distance_transformation (deprecated)

```
Calculate Physicochemical Distance Transformation (PDT) of protein sequence.
PDT describes how much a physicochemical property changes between residues
separated by l positions, for each lag l in the range 1..lag [12]. For each
AAindex property it is calculated as:

PDT_j(l) = sum_{i=1}^{L-l} (P_j(s_i) - P_j(s_i+l))^2 / (L - l)

Deprecated since v1.6.0, removed in 2.0: PDT_j(l) = 2 * s_j^2 * Geary_j(l)
exactly, where s_j^2 is the sample variance of property j over the sequence,
so it adds nothing to ``autocorrelation(method="geary")``.

where PDT_j(l) is the descriptor value for property j at lag l, P_j(s_i) is the
value of property j for the amino acid at position i, standardised over the 20
amino acids, and L is the length of the sequence. By default the same 8 AAindex
properties as the autocorrelation descriptors are used.
```

```
result = protpy.physicochemical_distance_transformation(protein_seq)
# Shape: 1 x 24
```

- **Columns:** PDT_CIDH920105_1, PDT_CIDH920105_2, PDT_CIDH920105_3, PDT_BHAR880101_1, PDT_BHAR880101_2, PDT_BHAR880101_3, ...
- **Parameters:** lag, properties
- **Use instead:** protpy.autocorrelation(seq, method="geary")

---

ncpr (deprecated)

```
Calculate the Net Charge Per Residue (NCPR) of protein sequence. NCPR is the
net charge of a protein sequence divided by its length [1], calculated as:

NCPR = (n+ - n-) / L

where n+ is the number of positive residues (K, R), n- the number of negative
residues (D, E) and L is the length of the sequence.

Deprecated since v1.6.0, removed in 2.0: NCPR is the Charge_NCPR column of
``charge_fractions``.
```

```
result = protpy.ncpr(protein_seq)
# Shape: 1 x 1
```

- **Columns:** NCPR
- **Parameters:** sequence only
- **Use instead:** protpy.charge_fractions(seq) # column Charge_NCPR

---

charge_density (deprecated)

```
Calculate the Charge Density of protein sequence. Charge density is the net
charge of a protein sequence at a given pH divided by its molecular weight [4],
calculated as:

ChargeDensity = NetCharge(pH) / MW

where NetCharge(pH) is calculated using the charge_distribution function and MW
using the molecular_weight function.

Deprecated since v1.6.0, removed in 2.0: use
``charge_distribution(sequence, ph, normalize="mass")`` (column
NetCharge_perDa); charge density correlates at r = 0.996 with NCPR.
```

```
result = protpy.charge_density(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `ChargeDensity`
- **Parameters:** `ph`
- **Use instead:** `protpy.charge_distribution(seq, normalize="mass")`

`hydrophobic_ratio` (deprecated)

```
Calculate the Hydrophobic Ratio of protein sequence. The hydrophobic ratio is
the percentage of hydrophobic residues in a protein sequence, as defined by
the Antimicrobial Peptide Database (APD) [7], calculated as:

HydrophobicRatio = N(LIVFMACW) / L * 100

where N(LIVFMACW) is the number of L, I, V, F, M, A, C and W residues and L
is the length of the sequence.

Deprecated since v1.6.0, removed in 2.0: it is a fixed linear map of the amino
acid composition (and close to the hydrophobic group of
``kmer_composition(k=1, alphabet=3)``).
```

```
result = protpy.hydrophobic_ratio(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `HydrophobicRatio`
- **Parameters:** sequence only
- **Use instead:** `protpy.kmer_composition(seq, k=1, alphabet=3)`

`half_life` (deprecated)

```
Calculate the estimated Half Life of protein sequence. The half-life is a
prediction of the time it takes for half of the amount of a protein in a cell
to disappear after its synthesis. It is estimated from the N-terminal residue
using the N-end rule [8, 9, 10], for 3 model systems: mammalian reticulocytes
(in vitro), yeast and E. coli (in vivo), using the values tabulated by ExPASy
ProtParam [6]. Values given as lower bounds in ProtParam (">20 hours" for yeast,
">10 hours" for E. coli) are reported as the bound. The E. coli value for an
N-terminal proline is unknown in ProtParam and returned as NaN. The estimate is
not applicable to N-terminally modified proteins.

Deprecated since v1.6.0, removed in 2.0: it depends only on the first residue
(which is Met in most proteins, so it is nearly constant) and the three
columns are perfectly correlated.
```

```
result = protpy.half_life(protein_seq)
# Shape: 1 x 3
```

- **Columns:** `HalfLife_Mammal_h`, `HalfLife_Yeast_h`, `HalfLife_Ecoli_h`
- **Parameters:** sequence only
- **Use instead:** — (no replacement)

`ivywrel_index` (deprecated)

```
Calculate the IVYWREL Index of protein sequence, the fraction of I, V, Y, W, R,
E and L residues [11].

Deprecated since v1.6.0, removed in 2.0: use ``thermostability_indices``
(column IVYWREL).
```

```
result = protpy.ivywrel_index(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `IVYWREL`
- **Parameters:** sequence only
- **Use instead:** `protpy.thermostability_indices(seq)`

`cvp_bias` (deprecated)

```
Calculate the Charged versus Polar Bias (CvP) of protein sequence,
%(D + E + K + R) - %(N + Q + S + T) [12].

Deprecated since v1.6.0, removed in 2.0: use ``thermostability_indices``
(column CvP_Bias).
```

```
result = protpy.cvp_bias(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `CvP_Bias`
- **Parameters:** sequence only
- **Use instead:** `protpy.thermostability_indices(seq)`

`flexibility` (deprecated)

```
Calculate the Flexibility (Flex) profile of protein sequence. Each amino acid
in the sequence is replaced by its normalised flexibility parameter (B-value)
and the values are averaged over a sliding window of 9 residues whose end
weights fall linearly to 0.25, as in Vihinen et al. [13], using the average
B-values (AAindex VINM940101, as in Biopython). The profile is summarised by
its mean, maximum, minimum and standard deviation.

Deprecated since v1.6.0, removed in 2.0: use
``hydrophobicity_profile(sequence, scale="vihinen_flexibility", window=9, edge=0.25)``,
which gives the same values.
```

```
result = protpy.flexibility(protein_seq)
# Shape: 1 x 4
```

- **Columns:** `Flex_mean`, `Flex_max`, `Flex_min`, `Flex_std`
- **Parameters:** `window`
- **Use instead:** `protpy.hydrophobicity_profile(seq, scale="vihinen_flexibility", window=9, edge=0.25)`

`linguistic_complexity` (deprecated)

```
Calculate the Linguistic Complexity (LC) of protein sequence. Linguistic
complexity describes how repetitive a sequence is, as the number of distinct
subwords (k-mers) it contains relative to the maximum possible number for a
sequence of its length [3]. It is calculated as:

LC = sum_{k=1}^{K} U_k / sum_{k=1}^{K} min(20^k, L - k + 1)

where U_k is the number of distinct k-mers in the sequence, L is the length of
the sequence and K is the longest subword length (by default L, i.e. all
subwords). Repetitive sequences have a low LC and random sequences an LC
close to 1.

Deprecated since v1.6.0, removed in 2.0: it costs O(L^2), saturates at
0.997-0.999 for real proteins, and the exact formula of [3] could not be
checked (some sources use a product of ratios). Use
``lempel_ziv_complexity`` or ``wootton_federhen_complexity``.
```

```
result = protpy.linguistic_complexity(protein_seq)
# Shape: 1 x 1
```

- **Columns:** `LinguisticComplexity`
- **Parameters:** `max_k`
- **Use instead:** `protpy.lempel_ziv_complexity(seq)`

`fft_descriptors` (deprecated)

```
Calculate Fast Fourier Transform (FFT) descriptors of protein sequence: the
binned power spectrum of a property signal [1].

Deprecated since v1.6.0, removed in 2.0: use
``spectral_descriptors(sequence, transform="fft", scale=scale, n=n_bins)``.
```

```
result = protpy.fft_descriptors(protein_seq)
# Shape: 1 x 10
```

- **Columns:** FFT_eiip_bin1, FFT_eiip_bin2, FFT_eiip_bin3, FFT_eiip_bin4, FFT_eiip_bin5, FFT_eiip_bin6, ...
- **Parameters:** scale, n_bins
- **Use instead:** protpy.spectral_descriptors(seq, transform="fft")

---

dct_descriptors (deprecated)

```
Calculate Discrete Cosine Transform (DCT) descriptors of protein sequence:
coefficients of the type-II DCT of a property signal [2].

Deprecated since v1.6.0, removed in 2.0: use
``spectral_descriptors(sequence, transform="dct", scale=scale, n=n_coeffs)``.
Since v1.6.0 the always-zero coefficient X_0 is dropped, so the coefficients
are 1..n_coeffs.
```

```
result = protpy.dct_descriptors(protein_seq)
# Shape: 1 x 10
```

- **Columns:** DCT_kyte_doolittle_1, DCT_kyte_doolittle_2, DCT_kyte_doolittle_3, DCT_kyte_doolittle_4, DCT_kyte_doolittle_5, DCT_kyte_doolittle_6, ...
- **Parameters:** scale, n_coeffs
- **Use instead:** protpy.spectral_descriptors(seq, transform="dct")

---

uversky_charge_hydropathy (deprecated)

```
Calculate the Uversky Charge-Hydropathy (UV) descriptors of protein sequence:
the mean rescaled hydropathy <H>, the absolute mean net charge <R>, the
distance HQ from the Uversky boundary and whether HQ < 0 [2] (see
charge_hydropathy).

Deprecated since v1.6.0, removed in 2.0: use ``charge_hydropathy(sequence)``
(column CH_HQ). UV_MeanH is (GRAVY + 4.5) / 9 and UV_MeanR is the absolute
NCPR of ``charge_fractions``.
```

```
result = protpy.uversky_charge_hydropathy(protein_seq)
# Shape: 1 x 4
```

- **Columns:** UV_MeanH, UV_MeanR, UV_HQ, UV_Unfolded
- **Parameters:** sequence only
- **Use instead:** protpy.charge_hydropathy(seq)

---

fold_index (deprecated)

```
Calculate the FoldIndex (FI) of protein sequence: the charge-hydropathy score
of the window centred on each residue [3] (see charge_hydropathy), summarised
by its mean and minimum, the fraction of residues with a negative score and
the longest run of such residues.

Deprecated since v1.6.0, removed in 2.0: use
``charge_hydropathy(sequence, window=51)``. Since v1.6.0 each window's score is
assigned to its centre residue; before, every residue of a negative window
was counted as disordered, which inflated the disordered fraction 2.6-7 times.
```

```
result = protpy.fold_index(protein_seq)
# Shape: 1 x 4
```

- **Columns:** FI_mean, FI_min, FI_frac_disordered, FI_longest_disordered
- **Parameters:** window
- **Use instead:** protpy.charge_hydropathy(seq, window=51)

`disorder_propensity` (deprecated)

```
Calculate the Disorder Propensity (DP) of protein sequence. Intrinsically
disordered proteins are enriched in disorder-promoting residues (A, R, G, Q,
S, P, E, K) and depleted in order-promoting residues (W, C, F, I, Y, V, L, N)
[4]. The descriptor gives the fraction of each, together with the mean value of
the TOP-IDP scale, a scale of amino acid propensity for intrinsic disorder [5],
calculated as:

DP_TOP_IDP_mean = sum_i TOP-IDP(s_i) / L

where TOP-IDP(s_i) is the TOP-IDP value of the amino acid at position i and L
is the length of the sequence. Higher values indicate a higher propensity for
disorder.

Deprecated since v1.6.0, removed in 2.0: all three values are fixed linear
maps of the amino acid composition and do not predict disorder. TOP-IDP is
available as a scale: ``gravy(sequence, scale="top_idp")`` or
``hydrophobicity_profile(sequence, scale="top_idp")``.
```

```
result = protpy.disorder_propensity(protein_seq)
# Shape: 1 x 3
```

- **Columns:** `DP_frac_disorder_promoting`, `DP_frac_order_promoting`, `DP_TOP_IDP_mean`
- **Parameters:** sequence only
- **Use instead:** `protpy.hydrophobicity_profile(seq, scale="top_idp")`

`transmembrane_features` (deprecated)

```
Calculate Transmembrane (TM) segment features of protein sequence. Putative
transmembrane segments are found from the Kyte-Doolittle hydropathy profile:
windows of 19 residues with a mean hydropathy above 1.6 indicate a likely
transmembrane segment [6], and overlapping windows are merged into segments.
The positive-inside rule states that the cytoplasmic side of a transmembrane
segment is enriched in positive residues [7]. The 4 output values are: the
number of segments; the maximum window hydropathy; the fraction of residues
inside a segment; and the positive-inside bias, the number of K and R residues
in the 15 residues before the first segment minus the number in the 15 residues
after it (0 if there is no segment). This is a hydropathy heuristic and not a
topology predictor.

Deprecated since v1.6.0, removed in 2.0: use
``hydrophobicity_profile(sequence, window=19, threshold=1.6, segments=True)``.
```

```
result = protpy.transmembrane_features(protein_seq)
# Shape: 1 x 4
```

- **Columns:** `TM_n_segments`, `TM_max_KD`, `TM_frac_in_TM`, `TM_positive_inside_bias`
- **Parameters:** `window`, `kd_threshold`
- **Use instead:** `protpy.hydrophobicity_profile(seq, window=19, threshold=1.6, segments=True)`

`signal_peptide_features` (deprecated)

```
Calculate Signal Peptide (SP) features of protein sequence. Signal peptides
have a positively charged N-terminal n-region followed by a hydrophobic
h-region [8]. The 2 output values are: the net charge of the first 5 residues
(K and R +1, D and E -1); and the maximum mean Kyte-Doolittle hydropathy over
8-residue windows within the first n_term residues. The von Heijne
cleavage-site weight matrix is not included.

Deprecated since v1.6.0, removed in 2.0: two crude N-terminal features without
a cleavage-site model are not a signal peptide predictor (and are confused by
N-terminal transmembrane segments).
```

```
result = protpy.signal_peptide_features(protein_seq)
# Shape: 1 x 2
```

- **Columns:** `SP_nregion_charge`, `SP_hregion_maxKD`
- **Parameters:** `n_term`
- **Use instead:** — (no replacement)

`functional_site_counts` (deprecated)

```
Calculate Functional Site (FS) counts of protein sequence. The number of each
functional sequence motif, and its density per 100 residues, is counted for:
N-glycosylation sites (PROSITE PS00001, N-{P}-[ST]-{P}), the C-terminal
endoplasmic reticulum retention signal (PS00014, [KRHQSA]-[DENQ]-E-L at the
C-terminus), the C-terminal peroxisomal targeting signal 1 (PS00342,
[STAGCN]-[RKH]-[LIVMAFY] at the C-terminus) [9] and tryptic cleavage sites
(after K or R, not before P or at the C-terminus) [10]. Overlapping matches
are counted.

Deprecated since v1.6.0, removed in 2.0: the four motifs are part of the
motif_composition library; use
``motif_composition(sequence, motifs=["N_glycosylation", "ER_retention", "PTS1", "tryptic_site"], per_100=True)``.
Since v1.6.0 a C-terminal K or R is not counted as a tryptic site.
```

```
result = protpy.functional_site_counts(protein_seq)
# Shape: 1 x 8
```

- **Columns:** `FS_N_glycosylation`, `FS_ER_retention`, `FS_PTS1`, `FS_tryptic_site`, `FS_N_glycosylation_per100`, `FS_ER_retention_per100`, ...
- **Parameters:** sequence only
- **Use instead:** `protpy.motif_composition(seq, motifs=["N_glycosylation", "ER_retention", "PTS1", "tryptic_site"], per_100=True)`

`antigenicity_surface` (deprecated)

```
Calculate the Antigenicity and Surface (Antigenicity) profiles of protein
sequence. Antigenic sites tend to be hydrophilic and exposed on the protein
surface. Each amino acid in the sequence is replaced by its value on the
Parker hydrophilicity scale, derived from HPLC peptide retention times [12],
and on the Janin scale of free energy of transfer from the inside to the
outside of a globular protein [13]. Each is averaged over a sliding window of 7
residues, as used by Parker et al., and the profile is summarised by its mean,
maximum, minimum and standard deviation.

Deprecated since v1.6.0, removed in 2.0: the values are exactly
``hydrophobicity_profile(sequence, scale="parker", window=7)`` and
``hydrophobicity_profile(sequence, scale="janin", window=7)``, and single-scale
profiles are close to random as epitope predictors.
```

```
result = protpy.antigenicity_surface(protein_seq)
# Shape: 1 x 8
```

- **Columns:** `PARKER_mean`, `PARKER_max`, `PARKER_min`, `PARKER_std`, `JANIN_mean`, `JANIN_max`, ...
- **Parameters:** `window`
- **Use instead:** `protpy.hydrophobicity_profile(seq, scale="parker", window=7)`

`cysteine_features` (deprecated)

```
Calculate Cysteine (Cys) features of protein sequence. Cysteines form
disulfide bonds and functional motifs such as the CxxC motif of thiol
oxidoreductases [14]. The 6 output values are: the number of cysteines; their
fraction; whether the number is even; the maximum number of disulfide bonds
(the number of cysteines // 2); the number of overlapping C-x-x-C motifs; and
the mean gap between consecutive cysteines (0 if there are fewer than 2).

Deprecated since v1.6.0, removed in 2.0: the count and fraction are the C
column of the amino acid composition, CxxC is a motif of motif_composition and
the mean spacing is (last - first) / (n - 1).
```

```
result = protpy.cysteine_features(protein_seq)
# Shape: 1 x 6
```

- **Columns:** `Cys_count`, `Cys_frac`, `Cys_even`, `Cys_max_disulfides`, `Cys_CxxC`, `Cys_mean_spacing`
- **Parameters:** sequence only
- **Use instead:** `protpy.amino_acid_composition(seq); motif_composition(seq, motifs=["CxxC_zinc_finger"])`

`correlation_function` (deprecated)

```
Calculate the correlation between 2 input amino acids based on the
physicochemical/structural properties from the protein sequence.

Deprecated since v1.6.0, removed in 2.0: it has no call sites in protpy.
```

```
result = protpy.composition.correlation_function()
# Returns: np.float64(1.16)
```

- **Parameters:** `Ri`, `Rj`, `properties`
- **Use instead:** — (no replacement)

`amphiphilic_correllation_function` (deprecated)

```
Calculate correlation value based on physicochemical property values
for input amino acids Ri and Rj.

Deprecated since v1.6.0, removed in 2.0: it has no call sites in protpy.
```

```
result = protpy.composition.amphiphilic_correllation_function()
# Returns: [np.float64(0.171), np.float64(0.47)]
```

- **Parameters:** `Ri`, `Rj`, `normalized_props`
- **Use instead:** — (no replacement)

Extensions to existing descriptors (v1.5.0)

These were opt-in in v1.5.0. In v1.6.0 the default pKa set became `"bjellqvist"`, `reduced_alphabet_composition` was merged into `kmer_composition(alphabet=)`, `ctd_` into `ctd()`, and the Eisenberg scale `"eisenberg"` was renamed `"eisenberg_arb1984"` (the default `hydrophobic_moment` table is `"eisenberg_jmb1984"`).

Function	New parameter(s)	Effect	Reference
`reduced_alphabet_composition`	`alphabet`, `k`	`alphabet="ifeature5"` with `k=1/2/3` gives GAAC (1 x 5), GDPC (1 x 25), GTPC (1 x 125); cp13/14/19/20 also accepted	Chen et al. 2018, 10.1093/bioinformatics/bty140
`conjoint_triad`	`gap`	`gap>0` gives KSCTriad, triads (s_i, s_{i+g+1}, s_{i+2g+2}) for g = 0..gap, columns suffixed `_gap{g}`	Chen et al. 2018; Shen et al. 2007, 10.1073/pnas.0607879104
`ctd_`	`property_set`	`"ifeature13"` adds 6 AAindex hydrophobicity partitions (13 properties, 1 x 273)	Dubchak et al. 1995, 10.1073/pnas.92.19.8700
`amino_acid_composition`, `dipeptide_composition`	`region`, `n`	`"nterm"`, `"cterm"` or `"split"` (N-terminal / middle / C-terminal blocks, prefixes `NT_`/`MID_`/`CT_`)	Kumar et al. 2006, 10.1074/jbc.M511061200
`isoelectric_point`, `charge_distribution`	`pka_scale`	`"emboss"`, `"bjellqvist"` (with residue-specific terminal pKa) or `"ipc_protein"`	Bjellqvist et al. 1993, 10.1002/elps.11501401163; Kozlowski 2016, 10.1186/s13062-016-0159-9
`gravy`	`scale`	average any single-value scale or AAindex code (column `GRAVY_{scale}`)	per scale
`hydrophobic_moment`	`scale`, `angle="helix"/"sheet"`	alternative hydrophobicity scale; named angles 100° / 160°	Eisenberg et al. 1984, 10.1073/pnas.81.1.140

Function	New parameter(s)	Effect	Reference
`extinction_coefficient`	`absorbance`	adds `Abs01_Reduced`, `Abs01_Oxidized` (A280 of a 1 g/L solution, ε / MW)	Pace et al. 1995, 10.1002/pro.5560041120

Utility Functions

`all_descriptors`

Compute a set of descriptors for a single protein sequence and return them concatenated into a single DataFrame.

Parameter	Type	Default	Description
`sequence`	str	—	Protein sequence
`params`	dict or None	`None`	Per-descriptor keyword arguments keyed by descriptor name
`show_progress`	bool	`False`	Display a `tqdm` progress bar naming each descriptor as it is computed
`output_file`	str or None	`None`	Path to save results; `None` disables file output
`save_each_sequence`	bool	`True`	Append this row to the output CSV immediately after computation; useful when calling in a loop
`descriptor_set`	str	`"classic"`	`"classic"` (about 1,400 columns: AAC, DPC, ProtParam scalars, Boman, charge, μH, Shannon entropy, motifs, PAAC, APAAC, Geary, CTriad, CTD, QSO with both matrices), `"extended"` (adds the low-dimensional descriptors) or `"all"` (adds CKSAAP, ASDC, distance pair, ACC, Moreau-Broto, Moran and SOCN)
`include`	list[str] or None	`None`	Extra registered descriptors to compute, e.g. `"tripeptide_composition"` (in no set)
`exclude`	list[str] or None	`None`	Descriptors to skip
`decimals`	int or None	`None`	Round every value; descriptors are not rounded otherwise (v1.6.0)

```
import protpy

seq = "MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDISIQEFKD"

# descriptor sets
df = protpy.all_descriptors(seq, descriptor_set="extended")
df = protpy.all_descriptors(seq, include=["kappa", "tripeptide_composition"], exclude=["ctd"],
                            params={"kappa": {"blob_sizes": (5,)}}, decimals=3)

# Basic usage
df = protpy.all_descriptors(seq)
# Shape: 1 x 1402 (the classic set; it was 1 x 9957 before v1.6.0, mostly tripeptide composition)

# With progress bar
df = protpy.all_descriptors(seq, show_progress=True)
# [Composition]    Amino acid composition  10%|███      |  3/30 [00:00<00:02]
```

```
# ...

# Save to file
df = protpy.all_descriptors(seq, output_file="out.csv")

# No file output
df = protpy.all_descriptors(seq, output_file=None)
```

`encode_dataset`

Compute all available descriptors for every sequence in a dataset (list or DataFrame) and return a DataFrame whose first column is a unique sequence identifier.

Parameter	Type	Default	Description
`sequences`	list[str] or pd.DataFrame	—	Sequences as a plain list or a two-column DataFrame from `read_sequences_from_file(..., id_col=...)`
`seq_col`	str	`"sequence"`	Column name containing sequences when input is a DataFrame
`params`	dict or None	`None`	Per-descriptor keyword arguments forwarded to each `all_descriptors` call
`show_progress`	bool	`False`	Show a per-descriptor `tqdm` bar for each individual sequence
`output_file`	str or None	`"descriptors.csv"`	CSV path; each completed row is flushed immediately so interrupted runs preserve progress
`save_each_sequence`	bool	`True`	Flush the CSV after each sequence; set `False` to skip all file output within this call
`strip_gaps`	bool	`False`	Remove gap characters (`-`) before encoding; useful for MSA-derived sequences

```
import protpy

# Plain list — identifiers auto-generated as seq1, seq2, …
seqs = [
    "MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGD",
    "ACDEFGHIKLMNPQRSTVWYACDEFGHIKLMNPQRSTVWY",
]
df = protpy.encode_dataset(seqs)
# df.columns[0] == "sequence_id"
# df["sequence_id"].tolist() == ["seq1", "seq2"]
# Shape: 2 x (1 + 10757)

# Named identifiers from a multi-column file
data = protpy.read_sequences_from_file(
    "thermostability.txt", seq_col="sequence", id_col="name"
)
df = protpy.encode_dataset(data, seq_col="sequence")
# df.columns[0] == "name"
# Shape: n_sequences x (1 + 10757)

# MSA sequences with gap characters
df = protpy.encode_dataset(seqs, strip_gaps=True)
```

`read_sequences_from_file`

Parse protein sequences from a FASTA, TXT, CSV, or TSV file. The format is inferred from the file extension.

Parameter	Type	Default	Description
`filepath`	str or os.PathLike	—	Path to the input file
`seq_col`	str, int, or None	`None`	Column containing sequences (name or zero-based index); ignored for plain FASTA and plain TXT
`id_col`	str, int, or None	`None`	Identifier column; when provided returns a DataFrame instead of a list
`has_header`	bool	`True`	Whether the first row of a delimited file is a header; columns must be selected by integer index when `False`

Supported formats:

Extension	Behaviour
`.fasta` / `.fa`	Standard FASTA; multi-line sequences joined; returns `list[str]`
`.txt` (no `seq_col`)	One raw sequence per non-blank line; returns `list[str]`
`.txt` (with `seq_col`)	Treated as delimited; separator auto-detected from first line
`.csv`	Comma-separated values
`.tsv`	Tab-separated values

When `id_col` is provided the function returns a `pd.DataFrame` with only the identifier and sequence columns; all other columns are dropped and rows with blank sequences are excluded. When `id_col` is omitted the function returns a plain `list[str]`.

Raises: `FileNotFoundError` if the file does not exist; `ValueError` for unsupported extensions or missing columns; `TypeError` if `seq_col` or `id_col` is not a str, int, or None.

```
import protpy

# FASTA
seqs = protpy.read_sequences_from_file("proteins.fasta")
# ['MKTAYIAKQRQISFVKSHFS...', 'ACDEFGHIKLM...', ...]

# Plain TXT (one sequence per line)
seqs = protpy.read_sequences_from_file("sequences.txt")

# CSV / TSV — by column name
seqs = protpy.read_sequences_from_file("data.csv", seq_col="sequence")

# CSV / TSV — by column index
seqs = protpy.read_sequences_from_file("data.tsv", seq_col=1)

# Multi-column file — keep id + sequence, drop everything else
df = protpy.read_sequences_from_file(
    "thermostability.txt", seq_col="sequence", id_col="name"
)
# Returns DataFrame:
#        name                          sequence
# 0  00000000  MTIKEMPQPKTFGELKNLPLLNTD...
# 1  11111111  KETSPIPQPKTFGPLGNLPLIDKD...

# Headerless file — select by index
seqs = protpy.read_sequences_from_file("seqs.csv", has_header=False, seq_col=0)
```

Descriptor Summary

Speed ratings reflect typical computation time for a single average-length protein (~500 residues) on a standard CPU:

Rating	Meaning
✓ Fast	< 1 ms — simple residue counting or scalar formula

Rating	Meaning
Moderate	1–50 ms — sliding window or O(n²) pass
Slow	> 50 ms — large feature space, iterative convergence, or many property lookups

> The table lists the registered descriptors (`protpy.list_descriptors()`), in registry order. "Descriptor set" is the smallest `all_descriptors` set that contains it (the sets are nested); deprecated functions are listed under *Deprecated in v1.6.0* above.

Function	Abbreviation	Output shape	Category	Descriptor set	Alias of	Speed
`amino_acid_composition`	AAComp	1 x 20	Composition	classic		✓ Fast
`dipeptide_composition`	DPComp	1 x 400	Composition	classic		✓ Fast
`gravy`	GRAVY	1 x 1	Composition	classic		✓ Fast
`aromaticity`	Aromaticity	1 x 1	Composition	classic		✓ Fast
`instability_index`	II	1 x 1	Composition	classic		✓ Fast
`isoelectric_point`	pI	1 x 1	Composition	classic		✓ Fast
`molecular_weight`	MW	1 x 1	Composition	classic		✓ Fast
`aliphatic_index`	AI	1 x 1	Composition	classic		✓ Fast
`extinction_coefficient`	ExtCoeff	1 x 2 [ExtCoeff_Reduced, ExtCoeff_Oxidized]	Composition	classic		✓ Fast
`boman_index`	BI	1 x 1	Composition	classic	`gravy`	✓ Fast
`charge_distribution`	ChargeDist	1 x 3	Composition	classic		✓ Fast
`hydrophobic_moment`	μH	1 x 2 [HydrophobicMoment_Mean, HydrophobicMoment_Max]	Composition	classic		✓ Fast
`shannon_entropy`	ShannonH	1 x 1	Complexity	classic		✓ Fast
`motif_composition`	MotifComp	1 x M (default 1 x 8; 1 x 2M with per_100)	Composition	classic		✓ Fast
`pseudo_amino_acid_composition`	PAAComp	1 x (20 + lamda) (default 1 x 50)	Composition	classic		Moderate
`amphiphilic_pseudo_amino_acid_composition`	APAAComp	1 x (20 + 2·lamda) (default 1 x 80)	Composition	classic		Moderate
`geary_autocorrelation`	GAuto	1 x (lag × n_properties) (default 1 x 240)	Autocorrelation	classic	`autocorrelation`	Moderate
`conjoint_triad`	CTriad	1 x 343·(gap+1) (default 1 x 343)	Conjoint Triad	classic		✓ Fast
`ctd`	CTD	1 x 147 (7 properties; 1 x 273 for ifeature13)	CTD	classic		Fast (single) / Moderate (all)
`quasi_sequence_order`	QSO	1 x (lag + 20) (default 1 x 50; 1 x 100 with distance_matrix='both')	Sequence Order	classic		Moderate
`natural_vector`	NV	1 x 60	Composition	extended		✓ Fast
`atomic_composition`	ATC	1 x 5	Composition	extended		✓ Fast
`cksaagp`	CKSAAGP	1 x 25·(gap+1) (default 1 x 150)	Composition	extended	`cksaap`	✓ Fast

Function	Abbreviation	Output shape	Category	Descriptor set	Alias of	Speed
charge_fractions	FCR	1 x 5	Physicochemical	extended		✓ Fast
kappa	Kappa	1 x 1	Physicochemical	extended		Moderate
sequence_charge_decoration	SCD	1 x 1	Physicochemical	extended		✓ Fast
hydrophobicity_profile	HP	1 x 4 (5 with threshold, 8 with segments)	Physicochemical	extended		✓ Fast
thermostability_indices	Thermo	1 x 2	Physicochemical	extended		✓ Fast
wootton_federhen_complexity	WF	1 x 3	Complexity	extended		✓ Fast
lempel_ziv_complexity	LZ	1 x 1	Complexity	extended		✓ Fast
spectral_descriptors	Spectral	1 x n (default 1 x 10)	Spectral	extended		✓ Fast
aggrescan	AGG	1 x 5 (1 x 12 with full=True)	Structural	extended		✓ Fast
charge_hydropathy	CH	1 x 1 (window=None) or 1 x 4	Structural	extended		✓ Fast
plaac	PLAAC	1 x 4	Structural	extended		✓ Fast
cksaap	CKSAAP	1 x 400·(gap+1) (default 1 x 2400), or G²·(gap+1) with an alphabet	Composition	all		Moderate
asdc	ASDC	1 x 400	Composition	all		✓ Fast
distance_pair	DP	1 x (G + G²·rng) (default 1 x 602)	Composition	all	cksaap	✓ Fast
auto_cross_covariance	ACC	1 x d²·lag (default 1 x 125)	Scales	all		✓ Fast
moreaubroto_autocorrelation	MBAuto	1 x (lag × n_properties) (default 1 x 240)	Autocorrelation	all	autocorrelation	Moderate
moran_autocorrelation	MAuto	1 x (lag × n_properties) (default 1 x 240)	Autocorrelation	all	autocorrelation	Moderate
sequence_order_coupling_number	SOCN	1 x lag (default 1 x 30; 1 x 60 with distance_matrix='both')	Sequence Order	all		Moderate
tripeptide_composition	TPComp	1 x 8000	Composition	opt-in	kmer_composition	Moderate
kmer_composition	kmer	1 x 20^k (default 1 x 400), or 1 x G^k with an alphabet	Composition	opt-in		Fast (k≤3) / Slow (k=4)
dde	DDE	1 x 400	Composition	opt-in	dipeptide_composition	Fast
autocorrelation	Auto	1 x (n_properties × lag) (default 1 x 240)	Autocorrelation	opt-in		Moderate
ctd_	CTD	1 x 21 (single property) / 1 x 147 (all 7 properties)	CTD	opt-in	ctd	Fast (single) / Moderate (all)
ctd_composition	CTD-C	1 x 3 (per property)	CTD	opt-in	ctd	✓ Fast

Function	Abbreviation	Output shape	Category	Descriptor set	Alias of	Speed
`ctd_transition`	CTD-T	1 x 3 (per property)	CTD	opt-in	`ctd`	✓ Fast
`ctd_distribution`	CTD-D	1 x 15 (per property)	CTD	opt-in	`ctd`	✓ Fast
`aa_scale_descriptors`	Scales	1 x d·domains (mode='mean') or 1 x L·d (mode='position')	Scales	opt-in		✓ Fast

References

The descriptors implemented in protpy are based on the following published methods:

Composition - Amino acid, dipeptide, and tripeptide composition: Nakashima, H., Nishikawa, K., & Ooi, T. (1986). The folding type of a protein is relevant to the amino acid composition. *Journal of Biochemistry*, 99(1), 153–162. - GRAVY: Kyte, J., & Doolittle, R. F. (1982). A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1), 105–132. - Aromaticity: Lobry, J. R., & Gautier, C. (1994). Hydrophobicity, expressivity and aromaticity are the major trends of amino-acid usage in 999 *Escherichia coli* chromosome-encoded genes. *Nucleic Acids Research*, 22(15), 3174–3180. - Instability index: Guruprasad, K., Reddy, B. V. B., & Pandit, M. W. (1990). Correlation between stability of a protein and its dipeptide composition: a novel approach for predicting in vivo stability of a protein from its primary sequence. *Protein Engineering*, 4(2), 155–161. - Isoelectric point: Bjellqvist, B., et al. (1994). The focusing positions of polypeptides in immobilized pH gradients can be predicted from their amino acid sequences. *Electrophoresis*, 14(1), 1023–1031. - Molecular weight: Gasteiger, E., et al. (2005). Protein identification and analysis tools on the ExPASy server. In *The Proteomics Protocols Handbook*, Humana Press, 571–607. - Isoelectric point, molecular weight, charge: Gasteiger, E., et al. (2005). *The Proteomics Protocols Handbook*, Humana Press, 571–607. - Secondary structure propensity: Chou, P. Y., & Fasman, G. D. (1974). Prediction of protein conformation. *Biochemistry*, 13(2), 222–245. - Aliphatic index: Ikai, A. J. (1980). Thermostability and aliphatic index of globular proteins. *Journal of Biochemistry*, 88(6), 1895–1898. - Extinction coefficient: Pace, C. N., et al. (1995). How to measure and predict the molar absorption coefficient of a protein. *Protein Science*, 4(11), 2411–2423. - Boman index: Boman, H. G. (2003). Antibacterial peptides: basic facts and emerging concepts. *Journal of Internal Medicine*, 254(3), 197–215. - Hydrophobic moment: Eisenberg, D., Weiss, R. M., & Terwilliger, T. C. (1982). The helical hydrophobic moment: a measure of the amphiphilicity of a helix. *Nature*, 299, 371–374. - Shannon entropy: Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423. - Pseudo amino acid composition (PseAAC): Chou, K.-C. (2001). Prediction of protein cellular attributes using pseudo-amino acid composition. *Proteins: Structure, Function, and Bioinformatics*, 43(3), 246–255. - Amphiphilic PseAAC (APseAAC): Chou, K.-C. (2005). Using amphiphilic pseudo amino acid composition to predict enzyme subfamily classes. *Bioinformatics*, 21(1), 10–19.

Autocorrelation - Moreau-Broto autocorrelation: Moreau, G., & Broto, P. (1980). The autocorrelation of a topological structure: A new molecular descriptor. *Nouveau Journal de Chimie*, 4, 359–360. - Moran autocorrelation: Moran, P. A. P. (1950). Notes on continuous stochastic phenomena. *Biometrika*, 37(1–2), 17–23. - Geary autocorrelation: Geary, R. C. (1954). The contiguity ratio and statistical mapping. *The Incorporated Statistician*, 5(3), 115–145. - AAIndex properties: Kawashima, S., & Kanehisa, M. (2000). AAindex: amino acid index database. *Nucleic Acids Research*, 28(1), 374.

Conjoint Triad - Liu, B., et al. (2008). Prediction of protein-protein interactions based on the naive Bayes classifier with amino acid composition features. *Biochemical and Biophysical Research Communications*, 368(2), 462–468. Doi: 10.1016/j.bbrc.2008.01.082.

CTD - Dubchak, I., et al. (1995). Prediction of protein folding class using global description of amino acid sequence. *PNAS*, 92(19), 8700–8704. - Dubchak, I., et al. (1999). Recognition of a protein fold in the context of the SCOP classification. *Proteins*, 35(4), 401–407.

v1.5.0 descriptors - Each v1.5.0 descriptor above lists its primary reference and DOI. BibTeX for every reference is in `docs/references.bib`.

Sequence Order - Grantham, R. (1974). Amino acid difference formula to help explain protein evolution. *Science*, 185(4154), 862–864. - Schneider, G., & Wrede, P. (1994). The rational design of amino acid sequences by artificial neural networks and simulated molecular evolution. *Biophysical Journal*, 66(2), 335–344. - Chou, K.-C. (2000). Prediction of protein subcellular locations by incorporating quasi-sequence-order effect. *Biochemical and Biophysical Research Communications*, 278(2), 477–483.

Back to top